



Tribhuvan University
Faculty of Humanities and Social Sciences

A Lab Report On
Advance Operating System (MCA502)

Submitted to
Department of Humanities and Social Science
Patan Multiple Campus

***In partial fulfillment of the requirements for the Master in Computer
Application***

Submitted by

Bishal Rijal

Roll no: 06

Class: MCA 1st Semester

Tu Register no: 6-2-1133-50-2019

Instructor

Tekendra Nath Yog

Table of Contents

Lab-1: Installation of oracle 21c DBMS software, Create Database schema for given application and insert the data into the database as required for the laboratory work.	1
Lab-2: Execute SQL data definition Language (DDL) commands (CREATE, ALTER, DROP, TRUNCATE, RENAME) to manage Database in the database.	8
Lab-3: Execute SQL data manipulation Language (DML) Commands (SELECT, INSERT, UPDATE, DELETE) to manipulate data in the database.	13
Lab-4: Execute SQL data control Language (DCL) commands (GRANT, REVOKE) to control the access of data in the database.	18
To execute SQL Data Control Language (DCL) commands, mainly GRANT and REVOKE, to control and manage user access and privileges on database objects. ..	
Data Control Language (DCL) acts as the security guard of a Database Management System (DBMS). It allows database administrators (DBAs) or data owners to dictate which users can execute specific statements (such as SELECT, INSERT, UPDATE, or DELETE) on specific tables, views, or schemas.	
The two main DCL commands are:	
Lab-5: Execute SQL transaction Control Language (TCL) commands (COMMIT, ROLLBACK, SAVEPOINT) to control transactions of data in the database	21
Lab-6: Write Queries using Arithmetic, relational and Logical operators to manipulate the data in the given database.	24
To write and execute SQL queries using Arithmetic, Relational, and Logical operators to manipulate and retrieve data from the given database.	

Lab-7: Illustrate enforcing integrity constraints (Domain, Key constraints (Primary/Foreign keys), not null, unique, default, Check) in the given database data.	27
Lab-8: Write queries to use Set operators (ANY, ALL, IN, INTERSECT, UNION) to perform different operations in the given database data.	30
Lab-9: Write queries using Date and Time functions, string functions and Arithmetic functions.	33
Lab-10: Implement queries using Aggregate functions, Ordering and Grouping (COUNT, SUM GROUPBY and HAVING.)	36
Lab-11: Write SQL queries to implement the various types of joins such as Inner and Outer Join.	39
Lab-12: Implement database views by creating, modifying, querying, and dropping views for efficient data access.	42
Lab-13: Implement SQL subqueries, including single-row, multiple-row, multiple-column, and correlated subqueries for data retrieval and manipulation.	45
Lab-14: Write a PL/SQL program to illustrate the Conditional Statements in PL/SQL.	48
Lab-15: Write a PL/SQL program to illustrate the Iterative Statements in PL/SQL.	51
Lab-16: Write a PL/SQL program to illustrate the Sequential Control in PL/SQL.	54
Lab-17: Write a PL/SQL program to demonstrate the use of an Implicit Cursor.	57
Lab-18: Write a PL/SQL program to demonstrate the use of an Implicit Cursor.	60
Lab-19: Write a PL/SQL program to illustrate Exception Handling using Predefined Exceptions.	63

Lab-20: Write a PL/SQL program to illustrate Exception Handling using User-Defined Exceptions.....	66
Lab-21: Create PL/SQL stored procedures using IN, OUT, and IN OUT parameters to illustrate modularity.	69
Lab-22: Write a PL/SQL procedures to illustrate the CRUD operations with the transaction control like commit, rollback in the database using stored procedure.	72
Lab-23: Create user-defined PL/SQL functions of your choice for the given database and demonstrate their use in SQL queries and PL/SQL blocks.	75
Lab-24: Write a PL/SQL program to create INSERT, UPDATE, and DELETE triggers for the given database.	78
Lab-25: Implement indexes and sequences for the given database and demonstrate their use in improving query performance and generating unique key values.....	81
Lab-26: Object and Object-Relational Databases (Oracle DB):.....	83
Create object types using Oracle Object Types.	83
Create object tables based on user-defined object types. Group of at most 2 members	83
Insert, update and delete objects from object tables.	83
Create nested object attributes.	83
Create collection data types (VARRAY and Nested Table).....	83
Implement object references (REF).	83
Create object methods (Member Functions and Procedures).	83
Query object tables using dot notation. - Implement inheritance using UNDER clause. -	
Write Object SQL queries for object retrieval.	83
Lab-27: Query Processing (Oracle DB):.....	87

Translate SQL queries into Relational Algebra.	87
Compare execution plans using EXPLAIN PLAN.....	87
Perform different JOIN operations.	87
Implement aggregation queries.....	87
Compare GROUP BY with HAVING.....	87
Compare Nested Loop Join and Hash Join.	87
Analyze ORDER BY using large datasets.....	87
Simulate pipelined query execution.....	87
Execute queries with indexes and without indexes.....	87
Compare query execution cost using AUTOTRACE.....	87
Lab-28: Query Optimization (Oracle DB):.....	91
Draw query trees for SQL statements.....	91
Convert SQL into relational algebra trees.	91
Optimize queries using heuristic rules.....	91
Compare optimized and non-optimized queries.	91
Analyze query execution plans.	91
Create indexes and observe optimization.	91
Measure execution time before and after indexing.....	91
Compare different joining algorithms.....	91
Perform cost-based optimization using Oracle Optimizer.....	91
Optimize complex nested queries.	91

Lab-29: Distributed Database Concepts (Oracle DB):	94
Install two database instances.	94
Create Database Links.	94
Execute distributed SQL queries.	94
Perform horizontal fragmentation.....	94
Perform vertical fragmentation.	94
Implement data replication.	94
Execute distributed joins.....	94
Simulate distributed transactions.	94
Demonstrate two-phase commit.	94
Lab-30: NOSQL (MongoDB):.....	97
Install MongoDB.	97
Create databases and collections.....	97
Insert JSON documents.....	97
Query documents.	97
Update documents.....	97
Delete documents.....	97
Perform aggregation pipeline operations.	97
Create indexes.....	97
Perform embedded document opera	97
Lab-31: Big Data with Hadoop.....	101

Lab-32: Enhanced Data Models.....	105
Project-33: CRUD Application on a Custom Database	109

Lab-1: Installation of oracle 21c DBMS software, Create Database schema for given application and insert the data into the database as required for the laboratory work.

Objective:

To set up Oracle Database 21c, connect to it through SQL Developer and the Oracle SQL Developer extension for VS Code, model a **Course Enrollment System (CES)** as an Entity-Relationship diagram, convert that model into physical tables, and load the sample records on which every later experiment depends.

Tools Used:

Oracle Database 21c (Enterprise/XE), SQLcl, SQL Developer, VS Code with Oracle SQL Developer for VS Code extension.

Database / Schema Used:

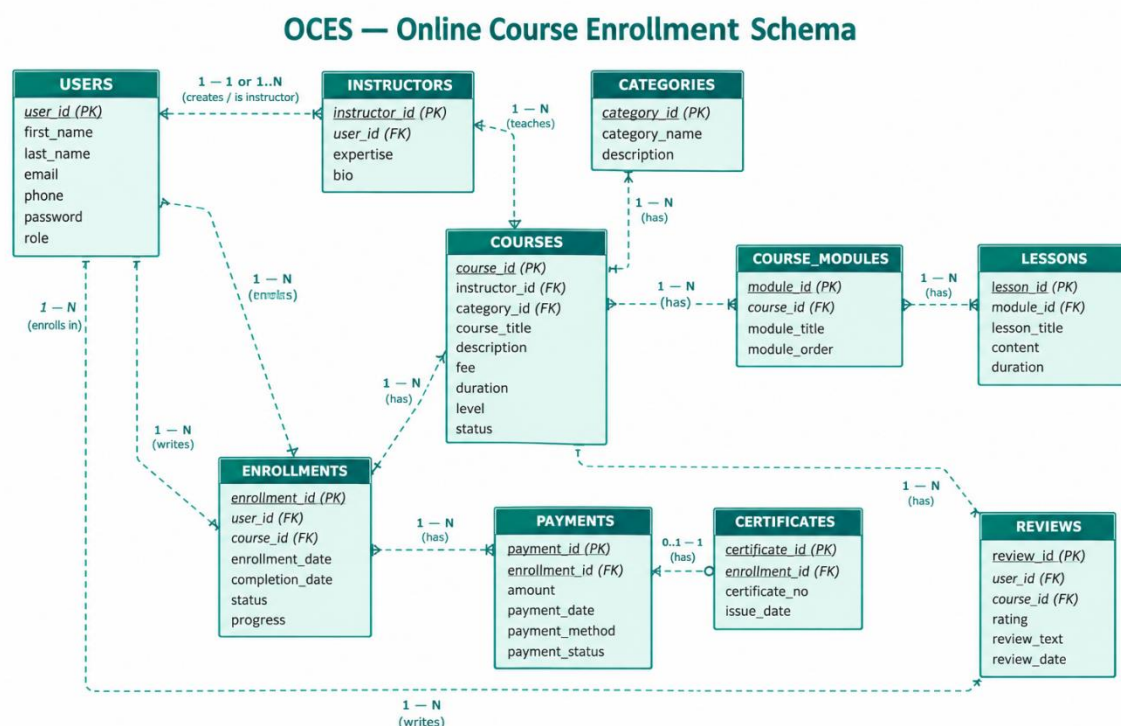


Fig. E-R diagram of the Online Course Enrollment (OCE) schema.

Theory:

Oracle Database 21c is Oracle's converged, object-relational DBMS, able to hold relational, JSON, spatial and graph data inside a single engine. Setting it up involves installing the software, letting the installer configure a TNS listener, and creating a

container database with at least one pluggable database (PDB) in which the application schema lives. Once the service is running, a client such as SQL Developer or the Oracle extension for VS Code connects to the PDB with an EZConnect string of the form `host:port/service_name`.

Schema design starts on paper rather than in the database. The university's requirements were first captured as an E-R model whose entities are DEPARTMENTS, PROGRAMS, STUDENTS, INSTRUCTORS, COURSES, SECTIONS and ENROLLMENTS. Mapping that model to the relational level turns each entity into a table, each attribute into a column and each relationship into a foreign key, so the design is already normalized before a single row is stored.

The physical schema is built with `CREATE TABLE` and then populated with `INSERT` statements. The baseline dataset — a small university offering a handful of programs with students enrolling in course sections — is intentionally small, so that every result printed later in this report can be verified by hand.

Query / Program and Output:

Installing and connecting to the database

Query:

```
SELECT banner_full FROM v$version;

SELECT sys_context('USERENV','CON_NAME') AS container FROM dual
```

Oracle Database 21c XE was installed with the official installer, keeping listener port 1521 and the default pluggable database XEPDB1. A dedicated schema user named CES was created inside XEPDB1, and a connection profile for it was saved in VS Code (Oracle SQL Developer extension). The two queries above confirmed the version banner and that the session was attached to the correct container.

Creating the schema (excerpt)

Syntax:

```
CREATE TABLE table_name (
column1 datatype constraint,
column2 datatype constraint,
...
);
```

Query:

```
CREATE TABLE departments (  
    dept_id    NUMBER(3) PRIMARY KEY,  
    dept_name  VARCHAR2(50) NOT NULL,  
    building   VARCHAR2(30)  
);  
  
CREATE TABLE programs (  
    program_id  NUMBER(4) PRIMARY KEY,  
    program_name VARCHAR2(60) NOT NULL,  
    dept_id     NUMBER(3) REFERENCES departments(dept_id),  
    duration_yrs NUMBER(1) DEFAULT 4  
);  
  
CREATE TABLE students (  
    student_id  NUMBER(6) PRIMARY KEY,  
    first_name  VARCHAR2(25) NOT NULL,  
    last_name   VARCHAR2(30) NOT NULL,  
    email       VARCHAR2(50) UNIQUE NOT NULL,  
    phone_no    VARCHAR2(15),  
    enroll_date DATE DEFAULT SYSDATE NOT NULL,  
    program_id  NUMBER(4) REFERENCES programs(program_id),  
    gpa         NUMBER(3,2)  
);  
  
CREATE TABLE instructors (  
    instructor_id NUMBER(5) PRIMARY KEY,  
    first_name    VARCHAR2(25) NOT NULL,
```

```

last_name  VARCHAR2(30) NOT NULL,
email      VARCHAR2(50) UNIQUE NOT NULL,
dept_id    NUMBER(3) REFERENCES departments(dept_id),
hire_date  DATE DEFAULT SYSDATE NOT NULL,
salary     NUMBER(9,2)
);

CREATE TABLE courses (
  course_id  NUMBER(5) PRIMARY KEY,
  course_code VARCHAR2(10) UNIQUE NOT NULL,
  course_title VARCHAR2(60) NOT NULL,
  credits    NUMBER(1) NOT NULL,
  dept_id    NUMBER(3) REFERENCES departments(dept_id)
);

CREATE TABLE sections (
  section_id  NUMBER(6) PRIMARY KEY,
  course_id   NUMBER(5) REFERENCES courses(course_id),
  instructor_id NUMBER(5) REFERENCES instructors(instructor_id),
  semester    VARCHAR2(15) NOT NULL,
  room_no     VARCHAR2(10),
  capacity    NUMBER(3) DEFAULT 40
);

CREATE TABLE enrollments (
  enroll_id  NUMBER(8) PRIMARY KEY,
  student_id NUMBER(6) REFERENCES students(student_id),
  section_id NUMBER(6) REFERENCES sections(section_id),

```

```
enroll_date DATE DEFAULT SYSDATE NOT NULL,  
grade      VARCHAR2(2),  
status     VARCHAR2(10) DEFAULT 'ENROLLED'  
);
```

Inserting the baseline dataset (excerpt)

Query:

```
INSERT INTO departments VALUES (1, 'Computer Science', 'Block A');  
INSERT INTO departments VALUES (2, 'Management', 'Block B');  
  
INSERT INTO programs VALUES (101, 'BSc Computer Science', 1, 4);  
INSERT INTO programs VALUES (102, 'BBA', 2, 4);  
  
INSERT INTO students VALUES  
(1001, 'Anita', 'Sharma', 'anita.sharma@uni.edu', '9841001001',  
DATE '2024-08-01', 101, 3.75);  
INSERT INTO students VALUES  
(1002, 'Rohan', 'Thapa', 'rohan.thapa@uni.edu', '9841001002',  
DATE '2024-08-01', 101, 3.40);  
  
INSERT INTO instructors VALUES  
(501, 'Deepak', 'Karki', 'deepak.karki@uni.edu', 1,  
DATE '2015-07-15', 95000);  
  
INSERT INTO courses VALUES  
(2001, 'CS101', 'Introduction to Programming', 3, 1);  
INSERT INTO courses VALUES  
(2002, 'CS102', 'Data Structures', 3, 1);  
  
INSERT INTO sections VALUES  
(3001, 2001, 501, 'Fall 2025', 'A-101', 40);
```

```

INSERT INTO sections VALUES
(3002, 2002, 501, 'Fall 2025', 'A-102', 35);

INSERT INTO enrollments VALUES
(4001, 1001, 3001, DATE '2025-08-20', NULL, 'ENROLLED');
INSERT INTO enrollments VALUES
(4002, 1002, 3001, DATE '2025-08-20', NULL, 'ENROLLED');
INSERT INTO enrollments VALUES
(4003, 1001, 3002, DATE '2025-08-21', NULL, 'ENROLLED');

COMMIT;

```

Verifying the load

Query:

```

EXEC DBMS_STATS.GATHER_SCHEMA_STATS('CES');

SELECT table_name, num_rows FROM user_tables ORDER BY table_name;

```

Output:

TABLE_NAME	NUM_ROWS
Courses	2
DEPARTMENTS	2
ENROLLMENTS	2
INSTRUCTORS	2
PROGRAMS	2
SECTIONS	2
STUDENTS	2

Screenshot (Execution Output):

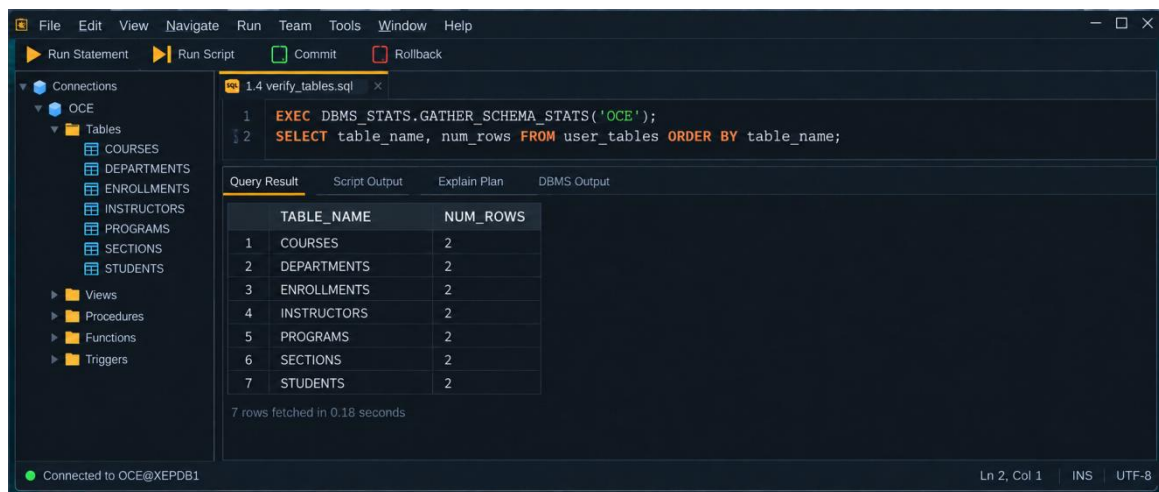


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

My first connection attempt failed with **ORA-12514** because I had typed the service name as **XE** instead of **XEPDB1**, so the listener did not know which database I wanted. After that fix the setup was straightforward. Drawing the E-R diagram before writing any SQL helped more than I expected: the foreign keys were already decided, so the **CREATE TABLE** scripts became mostly a translation exercise. The only real error came from loading **ENROLLMENTS** before **SECTIONS**, which raised **ORA-02291** (parent key not found) and made me reorder the script so that parent tables are always filled first.

With the schema and the baseline course-enrollment rows in place, the same data could be reused in every later experiment, which also made each output easy to predict and cross-check.

Lab-2: Execute SQL data definition Language (DDL) commands (CREATE, ALTER, DROP, TRUNCATE, RENAME) to manage Database in the database.

Objective:

To execute SQL Data Definition Language (DDL) commands (CREATE, ALTER, DROP, TRUNCATE, and RENAME) to manage database objects.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Data Definition Language (DDL) is a part of SQL (Structured Query Language) used to define and manage the structure of database objects such as tables, views, and schemas. DDL commands are mainly used to create, modify, rename, and remove database objects.

The major DDL commands used in this experiment are CREATE, ALTER, RENAME, TRUNCATE, and DROP.

Query / Program and Output:

CREATE: The CREATE command is used to create a new database object such as table.

Syntax:

```
CREATE TABLE table_name  
(column1 datatype, column2 datatype, ...);
```

Query:

```
CREATE TABLE students (  
    student_id NUMBER PRIMARY KEY,  
    student_name VARCHAR2(50),  
    age NUMBER,  
    address VARCHAR2(100),  
    email VARCHAR2(100)  
);
```

Output:

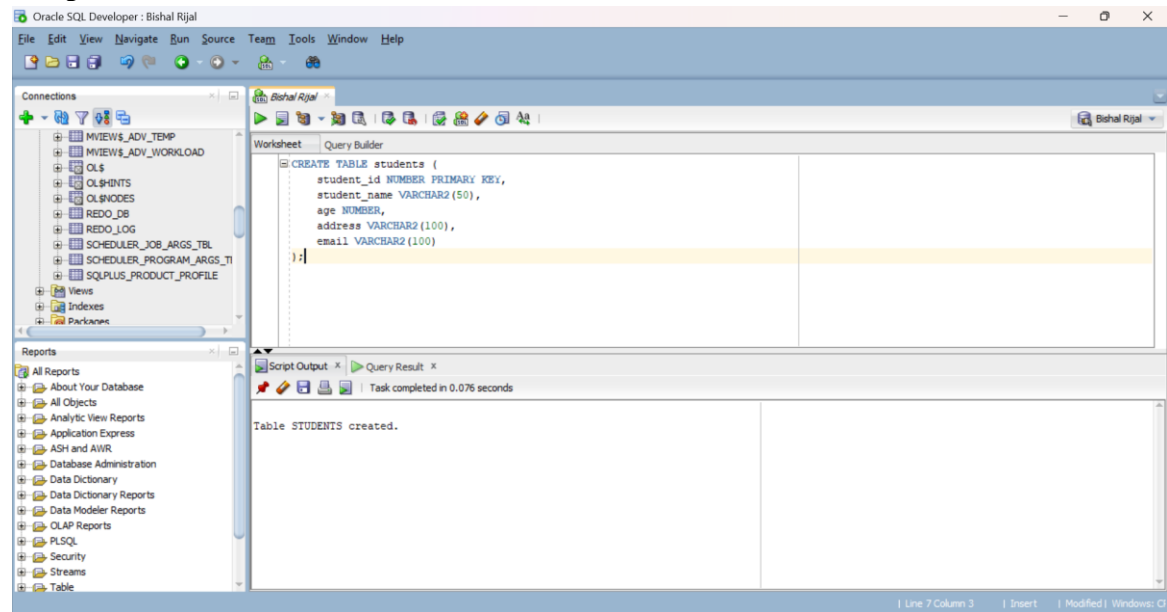


Figure 1: Creating the Students table using CREATE command

ALTER: The ALTER command is used to modify the structure of an existing table.

Syntax:

```
ALTER TABLE table_name  
ADD column_name datatype;
```

Query:

```
ALTER TABLE students  
ADD phone VARCHAR2 (15);
```

Output:

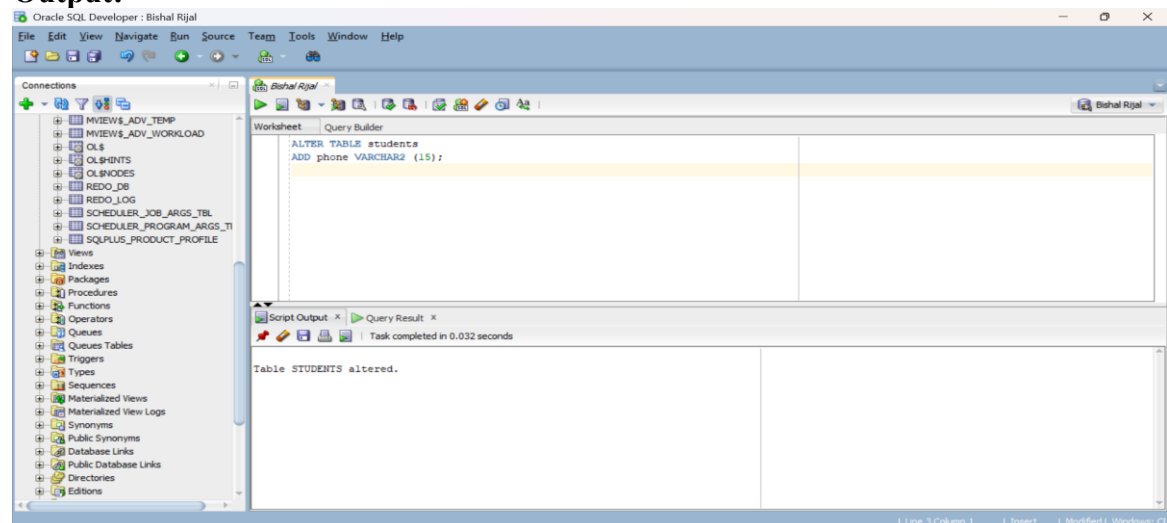


Figure 2: Adding a new column using ALTER command

RENAME: The RENAME command is used to change the name of an existing database object, such as a table.

Syntax:

```
RENAME old_table_name  
  
TO new_table_name;
```

Query:

```
RENAME students  
  
TO student_details;
```

Output

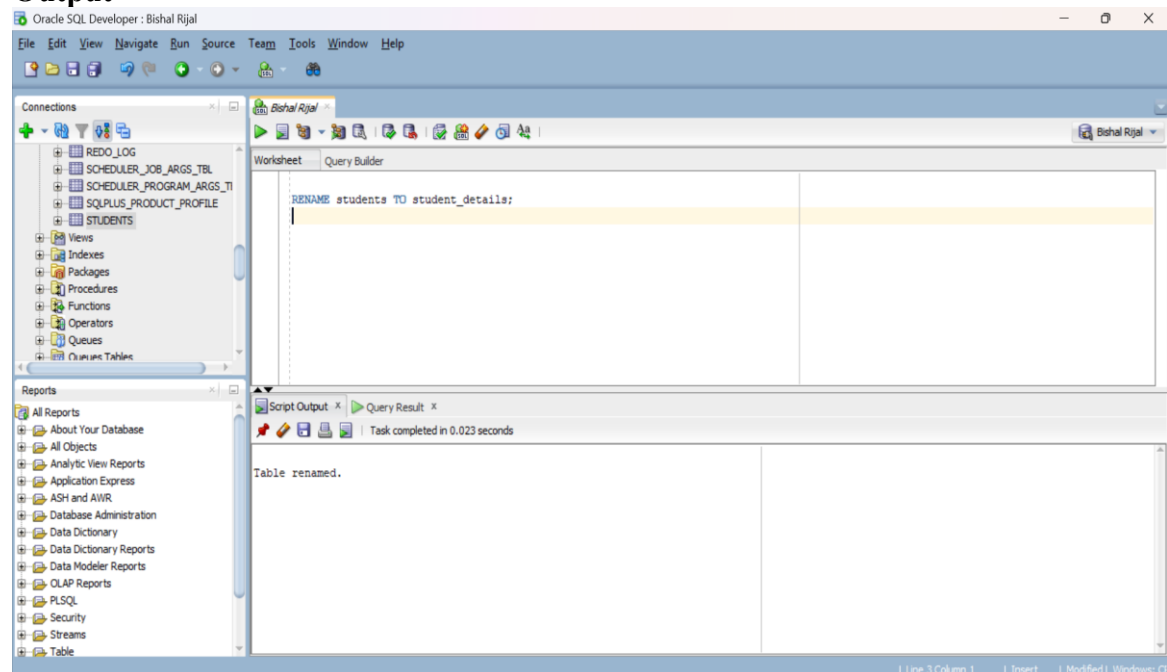


Figure 3: Renaming a column using ALTER command

TRUNCATE: The TRUNCATE command is used to remove all records from a table while keeping the table structure.

Syntax:

```
TRUNCATE TABLE table_name;
```

Query:

```
TRUNCATE TABLE student_details;
```

Output:

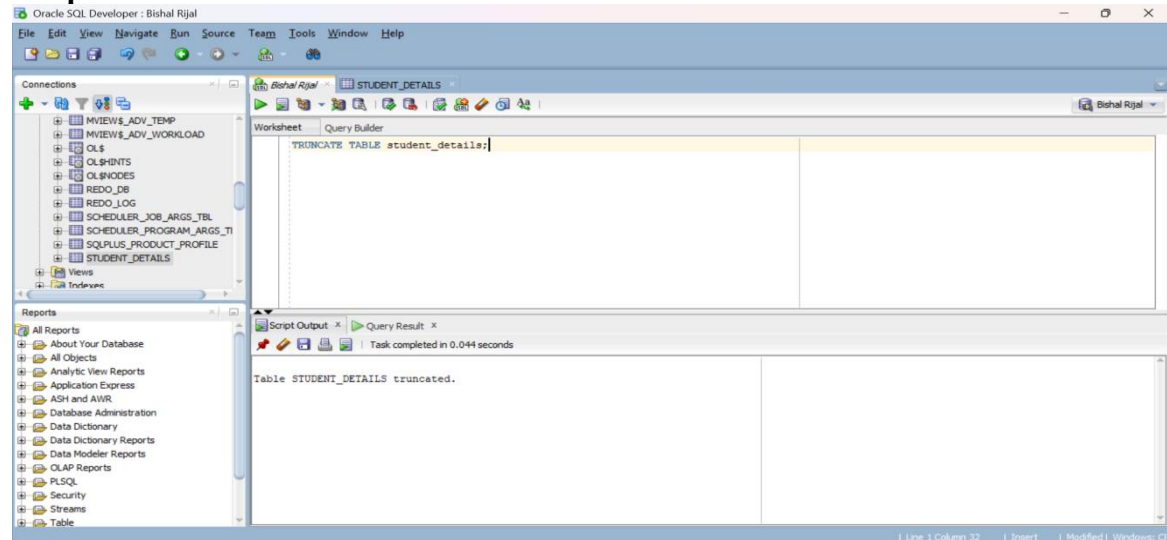


Figure 4: Removing all records using TRUNCATE command

DROP: The DROP command is used to permanently remove a database object, such as a table, including its structure and all the data stored in it.

Syntax:

```
DROP TABLE table_name;
```

Query:

```
DROP TABLE student_details;
```

Output:

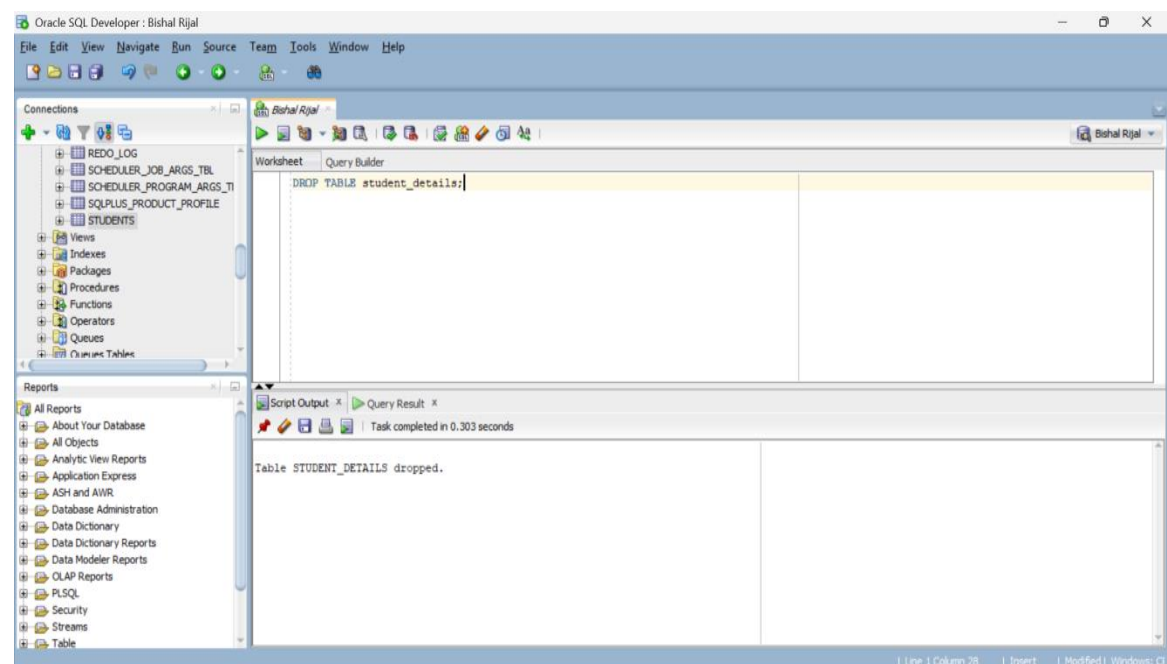


Figure 5: Removing the table using DROP command

Results:

The DDL commands CREATE, ALTER, RENAME, TRUNCATE, and DROP were successfully executed in Oracle SQL Developer. The commands were used to create, modify, rename, empty, and remove a database table.

Conclusions:

This experiment helped to understand the basic concepts and practical use of SQL DDL commands. The experiment demonstrated how database table structures can be created, modified, renamed, emptied, and deleted using different DDL commands.

Lab-3: Execute SQL data manipulation Language (DML) Commands (SELECT, INSERT, UPDATE, DELETE) to manipulate data in the database.

Objective:

To understand use of SQL Data Manipulation Language (DML) commands (SELECT, INSERT, UPDATE, DELETE) for manipulation of data in a Database.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Data Manipulation Language (DML) is a part of SQL used to manipulate and manage the data stored inside database tables. DML commands allow users to insert new records, retrieve existing records, modify existing records, and delete records from a table.

The major DML commands are SELECT, INSERT, UPDATE, DELETE.

Query / Program and Output:

INSERT: The INSERT command is used to add new records into a table.

Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Query:

```
INSERT INTO students
(student_id, student_name, age, address, email, phone) VALUES
(1, 'Bishal Rijal', 25, 'Chitwan', 'bishal@gmail.com', '9800000002');
```

Output:

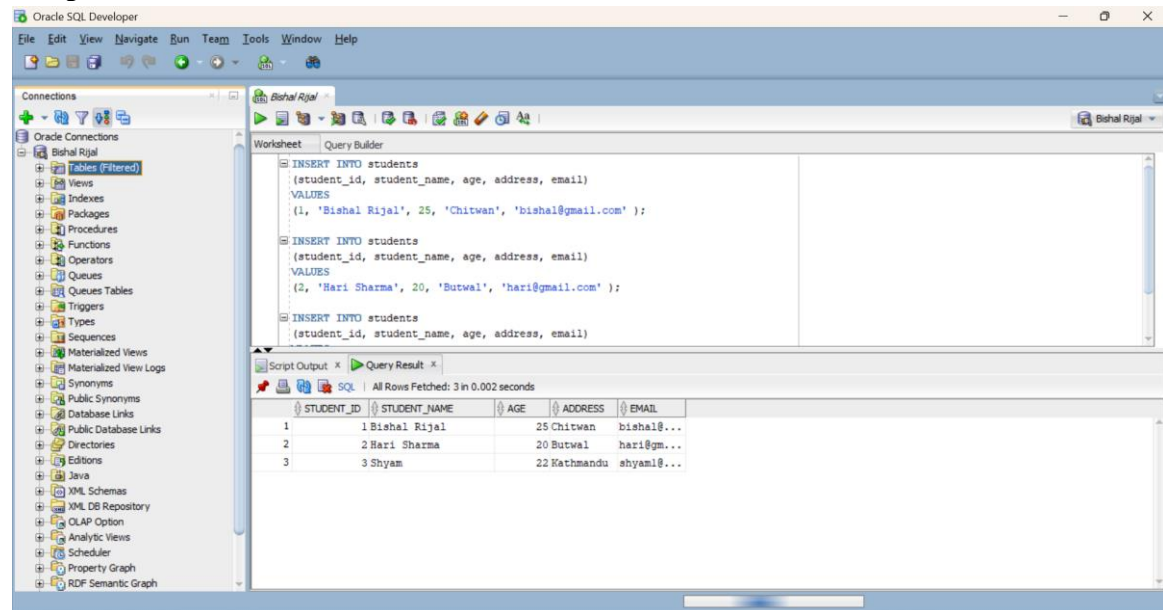


Figure 1: Inserting student records using INSERT command

UPDATE: The UPDATE command is used to modify existing data in a table.

Syntax:

```
UPDATE table_name
SET column = value, ...
WHERE condition;
```

Query:

```
UPDATE students
SET address = 'Kathmandu'
WHERE student_id = 1
```

Output:

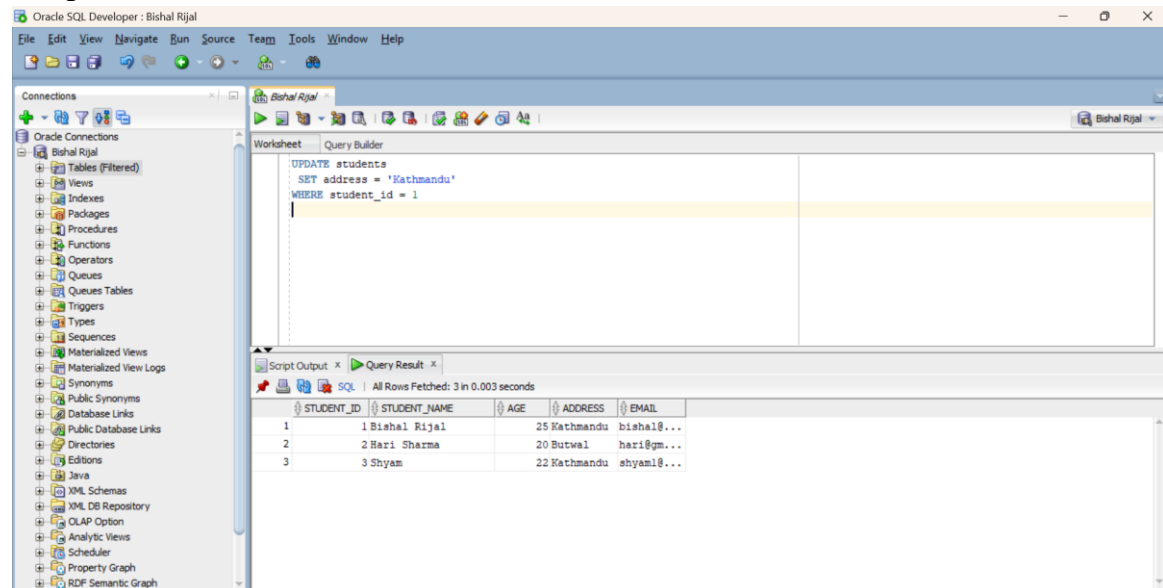


Figure 2: Updating student information using UPDATE command

DELETE: The DELETE command is used to remove existing records from a table.

Syntax:

DELETE FROM table_name WHERE condition;

Query:

DELETE FROM Students WHERE student_id = 2;

Output:

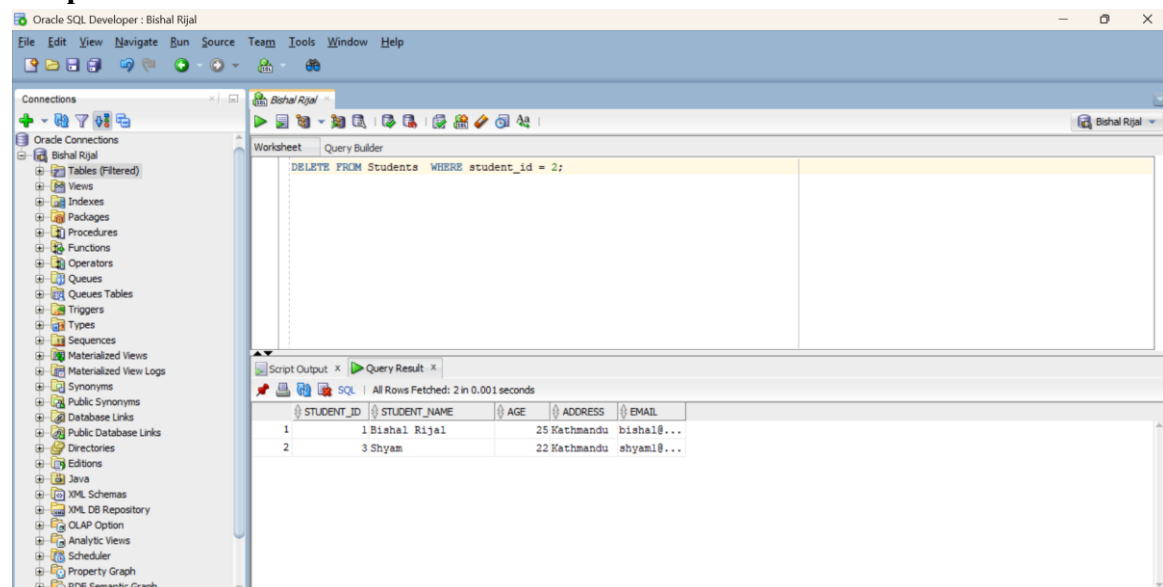


Figure 3: Deleting a student record using DELETE command

SELECT

Syntax:

```
SELECT column_list FROM table_name  
[WHERE condition] [ORDER BY column] [GROUP BY column];
```

Query:

```
SELECT * FROM students  
  
ORDER BY age;
```

Output:

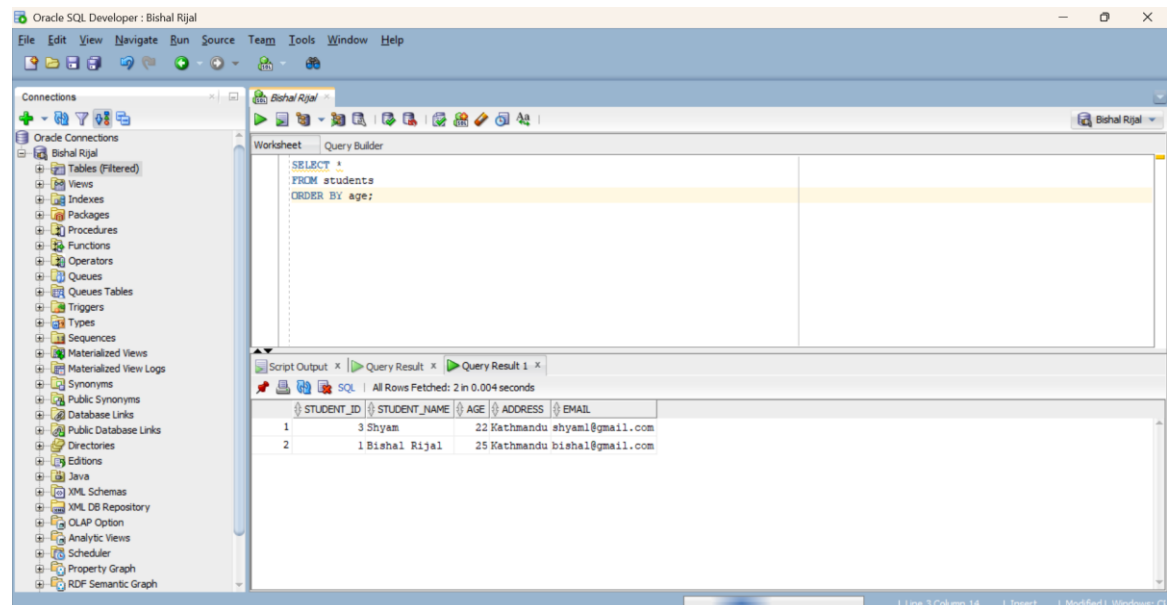


Figure 4 : Displaying student records using SELECT command

Results:

The DML commands were successfully executed on the students table using Oracle SQL Developer. The INSERT command was used to add new student records into the table by specifying values for student_id, student_name, age, address, email, and phone. The SELECT command was then executed to retrieve and display the inserted records and verify that the data had been stored correctly. The UPDATE command was used with the WHERE clause to identify a specific student record and modify its required information without affecting other records. Finally, the DELETE command was executed with a WHERE condition to remove the selected student record from the table. The results confirmed that DML commands can be used to perform the main data manipulation operations inserting, retrieving, modifying, and deleting records while the existing structure of the students table remains unchanged.

Conclusion:

This practical provided a clear understanding of how SQL commands interact with records stored in a relational database. By performing different operations on the `students` table, the role of each DML command and the importance of conditions such as `WHERE` became clear. Overall, the practical strengthened the understanding of basic database operations required for managing structured data.

Lab-4: Execute SQL data control Language (DCL) commands (GRANT, REVOKE) to control the access of data in the database.

Objective:

To execute SQL Data Control Language (DCL) commands, mainly GRANT and REVOKE, to control and manage user access and privileges on database objects.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension, a second test user account.

Theory:

Data Control Language (DCL) acts as the security guard of a Database Management System (DBMS). It allows database administrators (DBAs) or data owners to dictate which users can execute specific statements (such as SELECT, INSERT, UPDATE, or DELETE) on specific tables, views, or schemas.

The two main DCL commands are:

1. **GRANT** – Used to give privileges to a user.
2. **REVOKE** – Used to remove previously granted privileges from a user.

Query / Program and Output:

Creating a test user

Query:

```
CREATE USER student_viewer IDENTIFIED BY "View#2026";  
GRANT CREATE SESSION TO student_viewer;
```

GRANT

Syntax:

```
GRANT privilege [, privilege] ON object TO user | role;
```

Query:

```
GRANT SELECT ON students TO student_viewer;  
GRANT UPDATE (course) ON students TO student_viewer;
```

Verifying the grant

Query:

```
SELECT grantee, privilege, table_name, NULL AS column_name
FROM user_tab_privs_made
WHERE table_name = 'STUDENTS'
AND grantee = 'STUDENT_VIEWER'
UNION ALL
SELECT grantee, privilege, table_name, column_name
FROM user_col_privs_made
WHERE table_name = 'STUDENTS'
AND grantee = 'STUDENT_VIEWER';
```

Output:

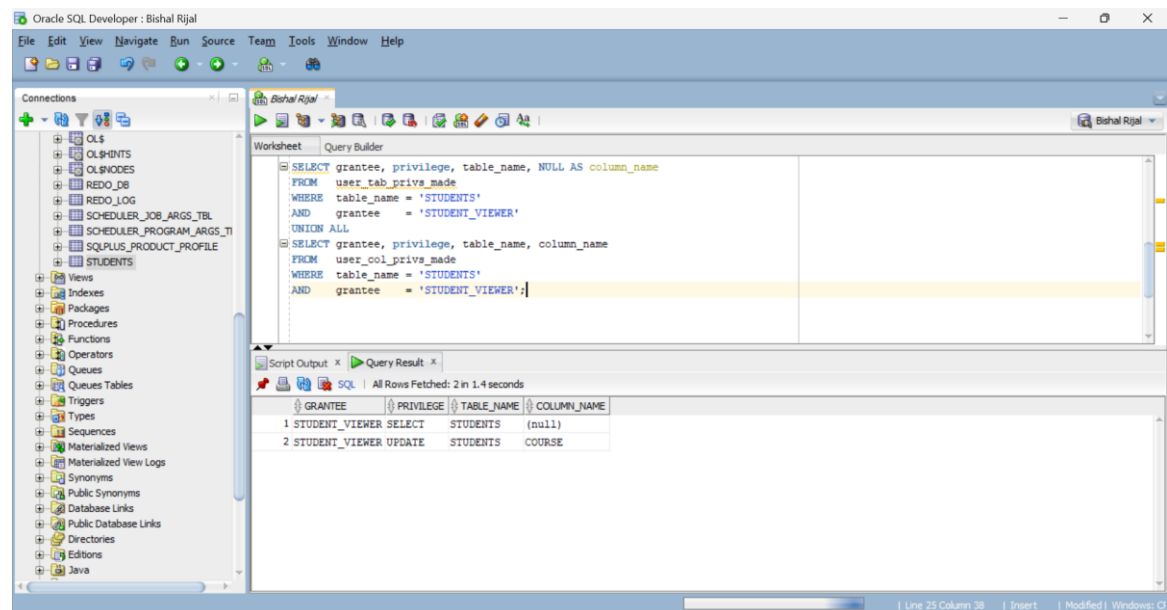


Figure 1: Granting privilege

REVOKE

Syntax:

```
REVOKE privilege [, privilege] ON object FROM user | role;
```

Query:

```
REVOKE UPDATE ON students FROM STUDENT_VIEWER;
```

Output:

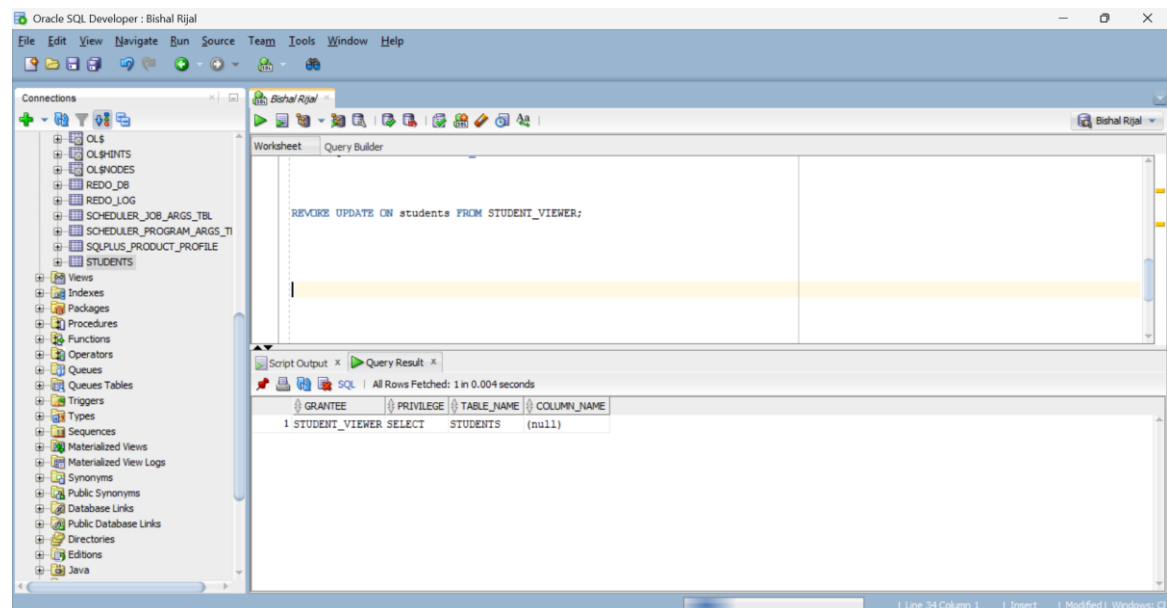


Figure 1: Granting privilege

Discussion & Conclusion:

In this task, a restricted Oracle user `student_viewer` was successfully created and granted `CREATE SESSION`, table-level `SELECT`, and column-level `UPDATE` (course) privileges on the `STUDENTS` table. The grants were verified using the `USER_TAB_PRIVS_MADE` and `USER_COL_PRIVS_MADE` data dictionary views, confirming that the user could read all student records and modify only the course column. The `UPDATE` privilege was then revoked with `REVOKE UPDATE ON students FROM STUDENT_VIEWER`, and a re-check of the same views showed that only `SELECT` remained. This exercise demonstrates the principle of least privilege in Oracle giving users only the access they need and being able to withdraw that access cleanly at any time.

Lab-5: Execute SQL transaction Control Language (TCL) commands (COMMIT, ROLLBACK, SAVEPOINT) to control transactions of data in the database

Group Type: Individual

Objective:

To execute SQL Transaction Control Language (TCL) commands such as COMMIT, ROLLBACK, and SAVEPOINT to control transactions and manage changes made to data in the database.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

TCL is a subset of SQL (Structured Query Language) that manages transactions in a database. It primarily consists of commands that control the behavior of transactions, which are sequences of operations performed as a single unit of work.

The three main TCL commands are COMMIT, ROLLBACK, and SAVEPOINT.

Query / Program and Output:

COMMIT: The COMMIT command is used to save all changes made during the current transaction to the database. Once a transaction is committed, the changes are permanent.

Query:

```
INSERT INTO students (student_id, student_name, course) VALUES (3, 'Ram Sharma',  
'BCA');  
  
COMMIT;
```

ROLLBACK: The ROLLBACK command is used to undo the changes made during the current transaction since the last COMMIT.

Query:

```
INSERT INTO students  
(students_id, students_name, course)  
VALUES  
(4, 'Shyam Thapa', 'MCA');  
ROLLBACK;
```

SAVEPOINT: SAVEPOINT creates a temporary point within a transaction. It allows us to roll back only the changes made after that particular point.

Syntax:

```
SAVEPOINT savepoint_name;  
ROLLBACK TO savepoint_name;
```

Query:

```
INSERT INTO students (student_id, student_name, course)  
VALUES (5, 'Hari KC', 'BIT');  
  
SAVEPOINT sp1;  
  
COMMIT;  
  
INSERT INTO students (student_id, student_name, course)  
VALUES (6, 'Gita Rai', 'BCA');  
  
ROLLBACK TO sp1;
```

Execution Output

The screenshot shows the Oracle SQL Developer interface. The 'Script Output' window displays the following SQL statements:

```
-- commit  
INSERT INTO students (student_id, student_name, course) VALUES (5, 'Ram Sharma', 'BCA');  
COMMIT;  
  
INSERT INTO students  
(student_id, student_name, course)  
VALUES  
(6, 'Shyam Thapa', 'MCA');  
ROLLBACK;  
  
INSERT INTO students (student_id, student_name, course)  
VALUES (5, 'Hari KC', 'BIT');  
-- Create a savepoint AFTER Hari's insert  
SAVEPOINT sp1;  
-- Insert Gita Rai  
INSERT INTO students (student_id, student_name, course)  
VALUES (6, 'Gita Rai', 'BCA');  
ROLLBACK TO sp1;
```

The 'Query Result' window shows the following data:

STUDENT_ID	STUDENT_NAME	COURSE
1	Bishal Rijal	Oracle Database
2	John Doe	Java Programming
3	Ram Sharma	BCA
4	Hari KC	BIT

Fig. Execution output of the statements above

Result:

The TCL commands were successfully executed on the STUDENTS table using Oracle SQL Developer. The COMMIT command was used to permanently save the inserted record of Ram Sharma into the table. The ROLLBACK command was then used to undo the insertion of Shyam Thapa, demonstrating that uncommitted changes can be cancelled. The SAVEPOINT command was used to create a temporary savepoint after inserting Hari KC. Another record for Gita Rai was inserted after the savepoint, and the ROLLBACK command was used to undo only Gita Rai's insertion while keeping Hari KC's record unchanged. The results confirmed that TCL commands can be used to control and manage transactions by permanently saving changes, cancelling uncommitted changes, and selectively rolling back changes to a specified savepoint.

Conclusion:

The TCL commands COMMIT, ROLLBACK, and SAVEPOINT were successfully executed on the students table. The COMMIT command was used to permanently save the changes, ROLLBACK was used to undo uncommitted changes, and SAVEPOINT was used to create a specific point for partially rolling back a transaction. Thus, the practical demonstrated how TCL commands can be used to control and manage transactions in an Oracle database.

Lab-6: Write Queries using Arithmetic, relational and Logical operators to manipulate the data in the given database.

Group Type: Individual

Objective:

To write and execute SQL queries using Arithmetic, Relational, and Logical operators to manipulate and retrieve data from the given database.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Arithmetic operators (+, -, *, /, %) perform mathematical calculations on numeric columns and derive new values directly in the SELECT list — they do not filter rows, they only compute expressions.

Relational operators (=, <>, !=, <, >, <=, >=, BETWEEN, IN, LIKE) compare two values and yield TRUE, FALSE, or UNKNOWN and they are used in the WHERE clause to filter rows that satisfy the comparison.

Logical operators (NOT, AND, OR evaluated in that order of precedence) join several conditions into one filter ,NOT reverses a condition, AND requires all conditions TRUE, OR requires any condition TRUE.

Query / Program and Output:

Arithmetic operators

Query:

```
SELECT student_id, student_name, age, age + 1 AS next_age  
FROM students;
```

Output:

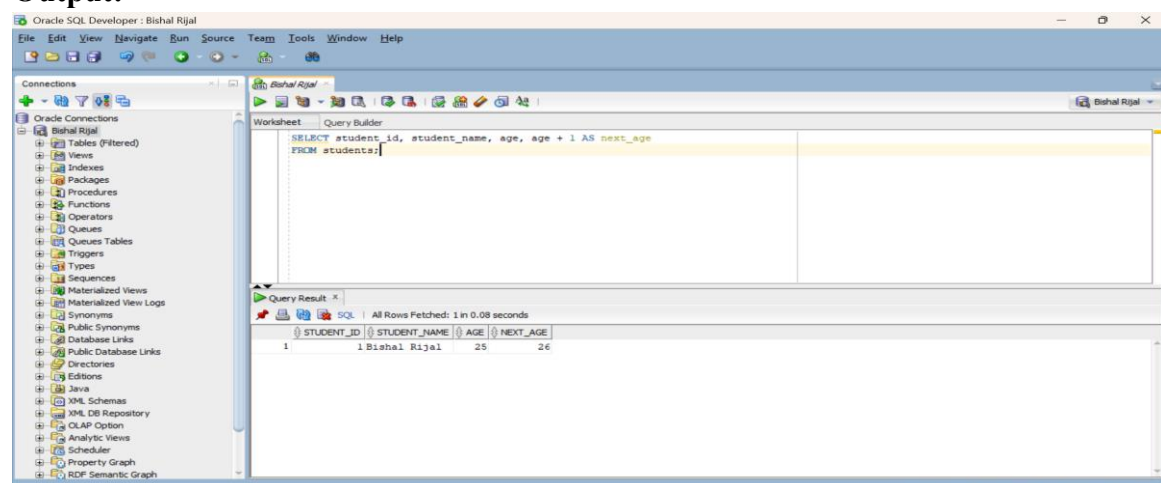


Figure 1: Execution of Arithmetic Operators for Age Calculation

Relational operators

Query:

```
SELECT student_id, student_name FROM students  
  
WHERE student_name <> 'Bishal Rijal';
```

Output:

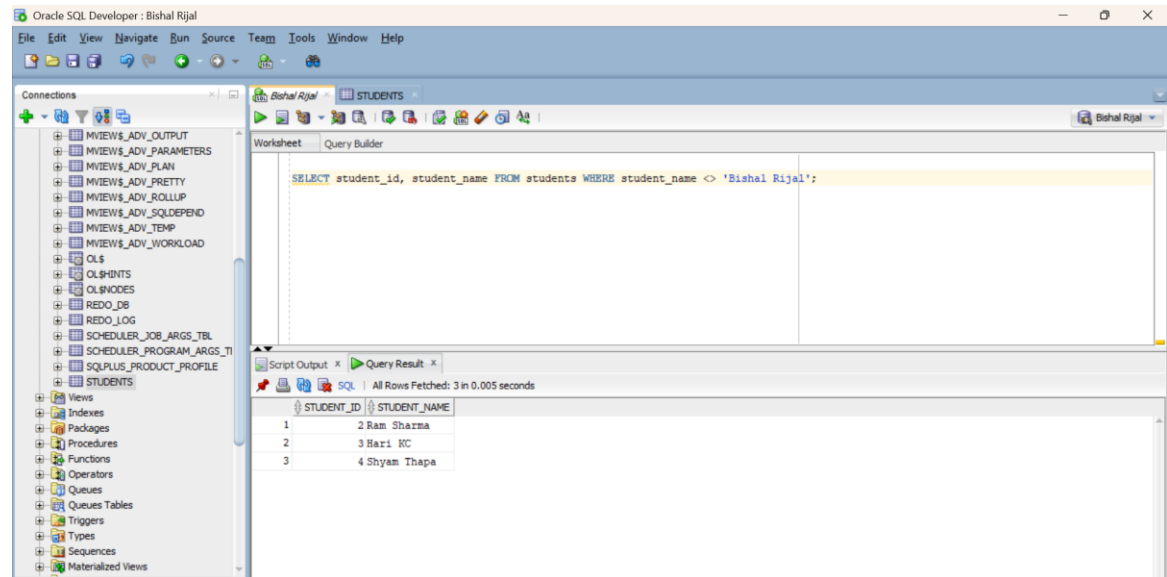


Figure 2: Execution of Relational Operators

Logical operators

Query:

```
SELECT student_id, student_name, age FROM students WHERE age > 20 OR age < 18  
  
ORDER BY age;
```

Output:

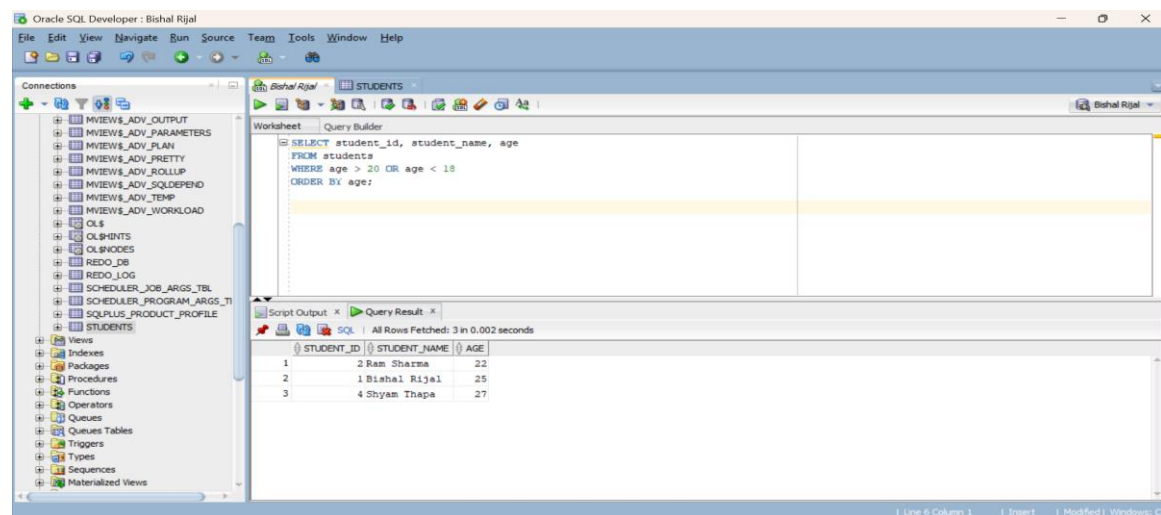


Figure 3: Logical Operators using OR Condition

Result:

The Arithmetic, Relational, and Logical operator queries were successfully executed on the students table using Oracle SQL Developer. The arithmetic operator was used to calculate the next age by adding 1 to the existing age values, while the relational operator $\diamond$ filtered and displayed students whose names were different from the specified name. The logical operator OR combined multiple age conditions to retrieve students either older than 20 or younger than 18, with the resulting records arranged in ascending order of age. These queries demonstrated how SQL operators can be used to perform calculations, compare values, and apply multiple conditions for effective data retrieval and manipulation.

Conclusion:

In this practical, Arithmetic (+), Relational ($\diamond$), and Logical (OR) operators were successfully executed on the students table in Oracle SQL Developer. The arithmetic operator derived a new column (next_age) in the SELECT list, the relational operator filtered rows using a comparison in the WHERE clause, and the logical operator combined multiple conditions into a single filter with sorted output. Thus, the practical demonstrated how these three operator types work together to manipulate and retrieve data efficiently in SQL.

Lab-7: Illustrate enforcing integrity constraints (Domain, Key constraints (Primary/Foreign keys), not null, unique, default, Check) in the given database data.

Objective:

To define and test domain, key (primary and foreign), NOT NULL, UNIQUE, DEFAULT and CHECK constraints so that invalid data is rejected by the database itself.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Integrity constraints are declarative rules stored with a table and checked by Oracle on every INSERT and UPDATE. A PRIMARY KEY enforces entity integrity by identifying each row uniquely and without NULLs; a FOREIGN KEY enforces referential integrity by requiring every child value to match an existing parent key; NOT NULL, UNIQUE, DEFAULT and CHECK enforce domain and business rules. Because the rules live in the database, no application can bypass them.

Query / Program and Output:

Key constraints

Query:

```
ALTER TABLE staff  
ADD CONSTRAINT staff_sup_fk FOREIGN KEY (supervisor_id)  
REFERENCES staff(staff_id);
```

NOT NULL / UNIQUE / DEFAULT / CHECK

Query:

```
ALTER TABLE staff MODIFY (first_name NOT NULL);  
ALTER TABLE staff ADD CONSTRAINT staff_phone_uk UNIQUE (phone_no);  
ALTER TABLE staff MODIFY (join_date DEFAULT SYSDATE);  
ALTER TABLE staff  
ADD CONSTRAINT staff_bonus_ck CHECK (bonus_pct BETWEEN 0 AND 0.30);
```

Testing the constraints

Query:

violates CHECK

```
INSERT INTO staff (staff_id, first_name, last_name, email, desig_code, salary, bonus_pct)
```

```
VALUES (401, 'Test', 'One', 'TONE', 'NURSE', 40000, 0.45);
```

violates UNIQUE

```
INSERT INTO staff (staff_id, first_name, last_name, email, desig_code, phone_no)
```

```
VALUES (402, 'Test', 'Two', 'TTWO', 'NURSE', '9851023456');
```

violates FOREIGN KEY

```
UPDATE staff SET supervisor_id = 999 WHERE staff_id = 212;
```

Output:

ATTEMPT	RESULT
Bonus of 45%	ORA-02290: check constraint (HMS.STAFF_BONUS_CK) violated
Duplicate phone number	ORA-00001: unique constraint (HMS.STAFF_PHONE_UK) violated
Unknown supervisor 999	ORA-02291: integrity constraint (HMS.STAFF_SUP_FK) violated - parent key not found

Screenshot (Execution Output):

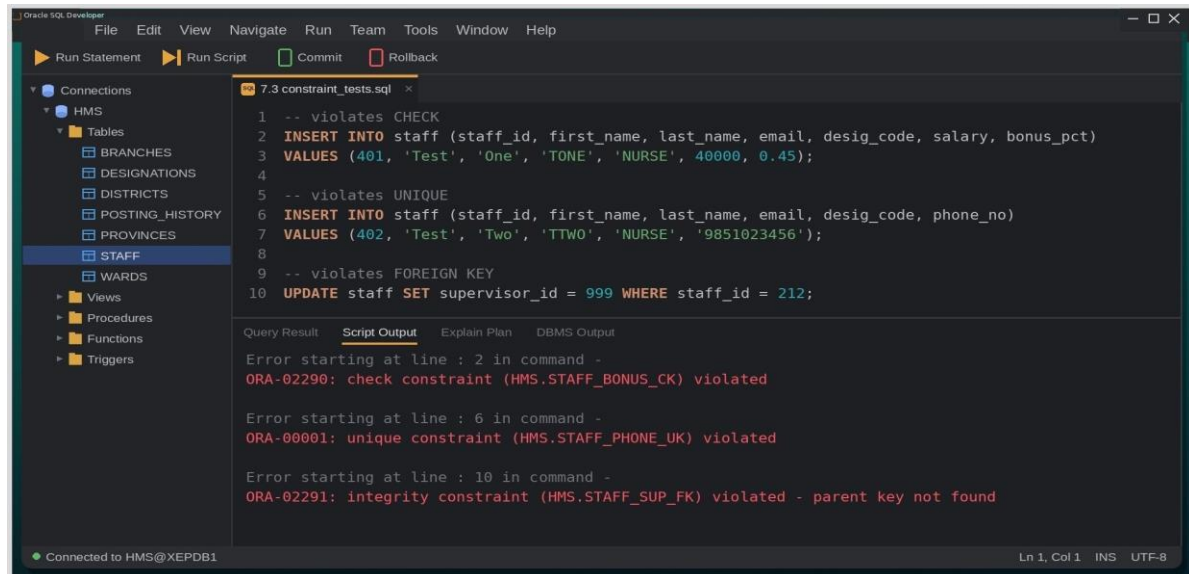


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

Writing the constraints took minutes; testing them was the valuable part. Each bad statement was rejected with a different error — ORA-02290 for the 45% bonus, ORA-00001 for the reused phone number and ORA-02291 for the non-existent supervisor — and because every constraint had a meaningful name, the message pointed straight at the rule that fired. The self-referencing foreign key on SUPERVISOR_ID models the hospital's reporting chain inside a single table.

Naming constraints explicitly is worth the extra typing; system-generated SYS_C names make error messages much harder to interpret.

Lab-8: Write queries to use Set operators (ANY, ALL, IN, INTERSECT, UNION) to perform different operations in the given database data.

Group Type: Individual

Objective:

To combine and compare query results with the IN, ANY, ALL, UNION and INTERSECT operators on the hospital database.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Set operators merge the results of two SELECT statements that return the same number of columns with compatible data types. UNION returns the distinct rows found in either query (UNION ALL keeps duplicates), INTERSECT returns only the rows present in both, and MINUS returns the rows of the first query that are missing from the second. IN, ANY and ALL, by contrast, test one value against a list or a subquery result inside a WHERE clause.

Query / Program and Output:

IN

Query:

```
SELECT first_name, ward_id FROM staff
WHERE ward_id IN (20, 40);
```

ANY / ALL

Query:

```
SELECT first_name, salary FROM staff
WHERE salary > ANY (SELECT salary FROM staff WHERE ward_id = 50);

SELECT first_name, salary FROM staff
WHERE salary > ALL (SELECT salary FROM staff WHERE ward_id = 50);
```

UNION

Query:

```
SELECT staff_id FROM staff WHERE ward_id = 50  
  
UNION  
  
SELECT staff_id FROM staff WHERE desig_code = 'NURSE';
```

Output:

STAFF_ID
206
208
209
210
211
212

INTERSECT

Query:

```
SELECT staff_id FROM staff WHERE ward_id = 50  
  
INTERSECT  
  
SELECT staff_id FROM staff WHERE desig_code = 'NURSE';
```

Output:

STAFF_ID
210

Screenshot (Execution Output):

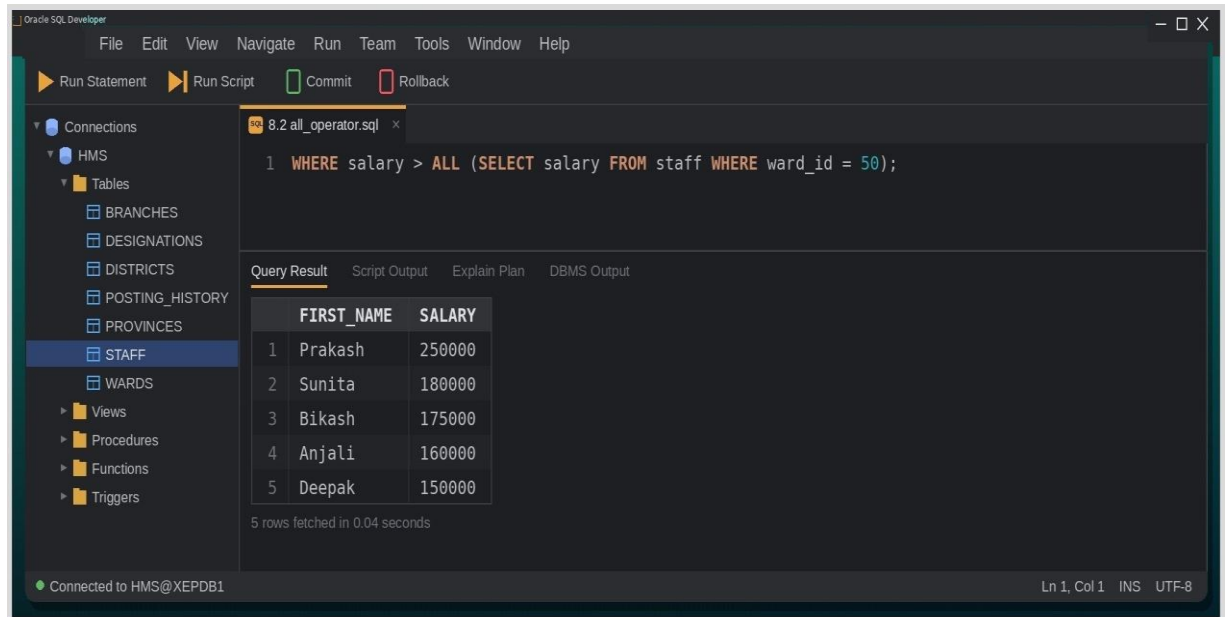


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

ANY and ALL look alike but ask opposite questions. With Emergency salaries ranging from 38000 to 98000, '> ANY' only had to beat the lowest value, so eleven of the twelve staff qualified, whereas '> ALL' had to beat the highest and left just the five senior staff earning more than 98000. UNION and INTERSECT on the same pair of queries were equally revealing: six distinct IDs in total, but only Manisha (210) is both a nurse and posted in Emergency.

Set operators make it easy to compare two groups of rows directly, without writing joins or checking the overlap by hand.

Lab-9: Write queries using Date and Time functions, string functions and Arithmetic functions.

Group Type: Individual

Objective:

To apply Oracle's single-row date, character and numeric functions in SQL queries.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Single-row functions return one result for every input row. Date functions such as SYSDATE, ADD_MONTHS, MONTHS_BETWEEN and TO_CHAR work on DATE values; character functions such as UPPER, INITCAP, SUBSTR, LPAD, LENGTH and TRIM transform strings; numeric functions such as ROUND, TRUNC, CEIL, FLOOR and MOD perform calculations on numbers.

Query / Program and Output:

Date functions

Query:

```
SELECT first_name, join_date,  
ROUND(MONTHS_BETWEEN(SYSDATE, join_date)/12, 1) AS service_years,  
TO_CHAR(join_date, 'DD Mon YYYY') AS joined_on  
FROM staff  
WHERE ward_id = 10;
```

String functions

Query:

```
SELECT UPPER(last_name) AS surname,  
INITCAP(LOWER(email)) AS mail_id,  
SUBSTR(phone_no, 1, 3) AS operator_code,  
LPAD(desig_code, 10, '*') AS padded_desig  
FROM staff  
WHERE ward_id = 20;
```

Arithmetic (numeric) functions

Query:

```
SELECT first_name, salary,  
ROUND(salary/30, 2) AS daily_rate,  
TRUNC(salary/30) AS trunc_rate,  
MOD(staff_id, 3) AS id_mod_3  
FROM staff  
WHERE ward_id = 40;
```

Output:

FIRST_NAME	SALARY	DAILY_RATE	TRUNC_RATE	ID_MOD_3
Deepak	150000	5000	5000	0
Pooja	46000	1533.33	1533	1

Screenshot (Execution Output):

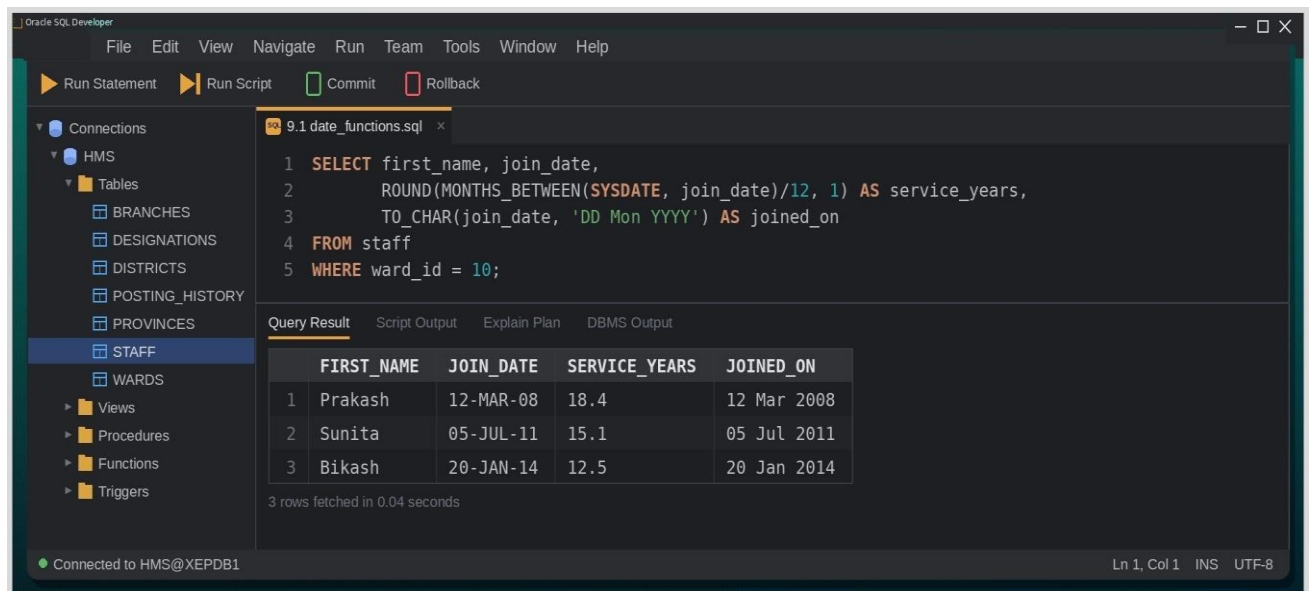


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

The difference between ROUND and TRUNC only showed up with Pooja's salary: 46000/30 is 1533.33 when rounded but 1533 when truncated, while Deepak's salary

divides evenly and gives 5000 either way. The TO_CHAR format model was the fiddliest part — 'Mon' and 'MON' produce different capitalisation, and I had to rerun the query before 'DD Mon YYYY' came out exactly right.

These functions let the query itself format and calculate values, so the application receives data that is ready to display.

Lab-10: Implement queries using Aggregate functions, Ordering and Grouping (COUNT, SUM GROUPBY and HAVING.)

Group Type: Individual

Objective:

To summaries data with aggregate functions (COUNT, SUM, AVG, MAX), sort results with ORDER BY and form groups with GROUP BY and HAVING.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Aggregate functions — COUNT, SUM, AVG, MIN and MAX — reduce a set of rows to a single value. GROUP BY divides the rows into groups that share the same value in one or more columns, so the aggregate is calculated once per group. WHERE filters individual rows before grouping takes place, whereas HAVING filters whole groups after the aggregates have been computed.

Query / Program and Output:

Basic aggregation

Query:

```
SELECT COUNT(*) AS total_staff, SUM(salary) AS monthly_payroll,  
MAX(salary) AS top_salary  
FROM staff;
```

GROUP BY with ORDER BY

Query:

```
SELECT ward_id, COUNT(*) AS staff_count, SUM(salary) AS ward_payroll  
FROM staff  
GROUP BY ward_id  
ORDER BY ward_payroll DESC;
```

HAVING

Query:

```
SELECT ward_id, ROUND(AVG(salary),2) AS avg_salary
FROM staff
GROUP BY ward_id
HAVING AVG(salary) > 100000
ORDER BY avg_salary DESC;
```

Output:

WARD_ID	AVG_SALARY
10	201666.67
20	101000

Screenshot (Execution Output):

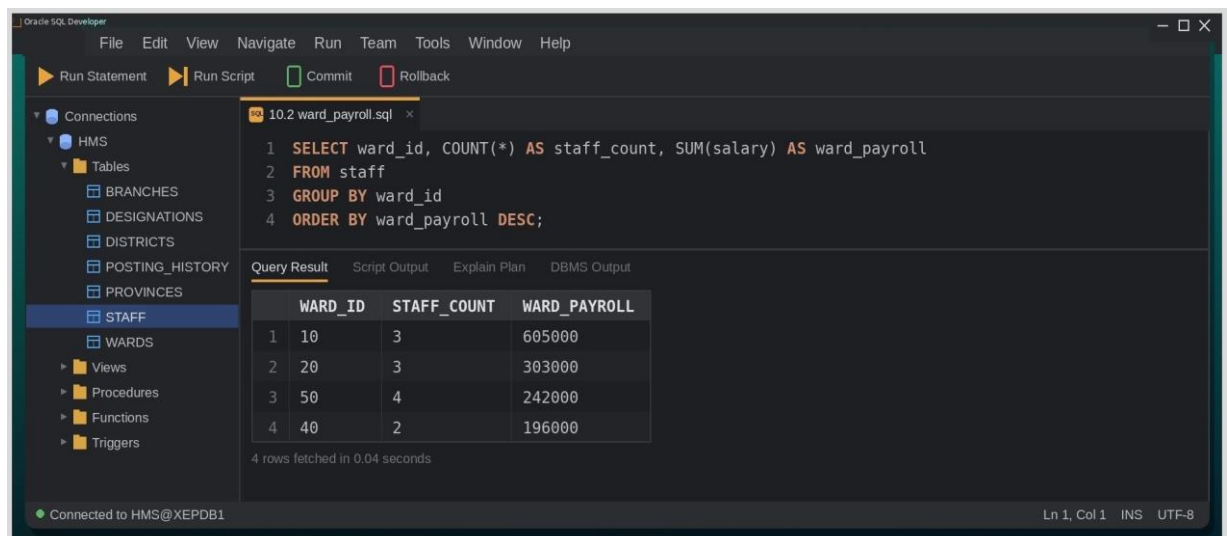


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

My first version of 10.3 put $\text{AVG}(\text{salary}) > 100000$ in the WHERE clause and failed with ORA-00934: group function is not allowed here, which made the WHERE/HAVING distinction impossible to forget. The grouped payroll in 10.2 also served as a sanity check: the four ward totals add up to 1,346,000, exactly the figure $\text{SUM}(\text{salary})$ returned

in 10.1. Emergency has the largest team but only the third-highest payroll, something a plain row count would never reveal. Grouping turns raw rows into information that answers management questions directly.

Lab-11: Write SQL queries to implement the various types of joins such as Inner and Outer Join

Group Type: Individual

Objective:

To retrieve data from several tables using inner joins, outer joins and a self join.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Database / Schema Used:

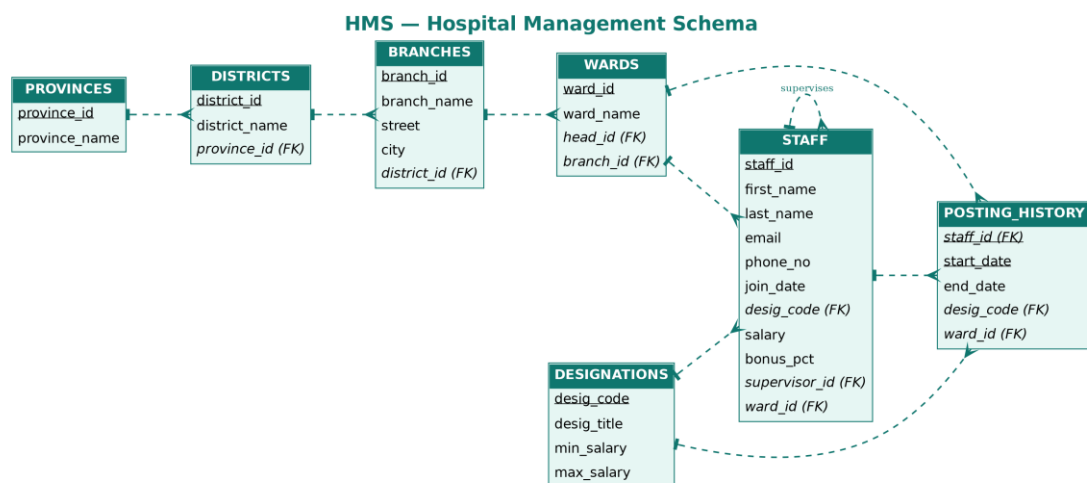


Fig. E-R diagram of the Hospital Management (HMS) schema.

Theory:

A join combines rows of two or more tables wherever a join condition — normally a foreign key matching a primary key — is satisfied. An INNER JOIN keeps only the matching pairs. LEFT, RIGHT and FULL OUTER JOINS also keep the unmatched rows of the left table, the right table or both, filling the columns of the missing side with NULL. A self join joins a table to itself through two different aliases.

Query / Program and Output:

INNER JOIN

Query:

```
SELECT s.first_name, s.last_name, w.ward_name
FROM staff s
JOIN wards w ON s.ward_id = w.ward_id
ORDER BY w.ward_name, s.first_name;
```

LEFT OUTER JOIN

Query:

```
SELECT w.ward_name, s.first_name  
FROM wards w  
LEFT JOIN staff s ON w.ward_id = s.ward_id  
ORDER BY w.ward_name, s.first_name;
```

Physiotherapy (ward 80) has no staff yet, but the LEFT JOIN still lists it, with NULL in FIRST_NAME.

Self join

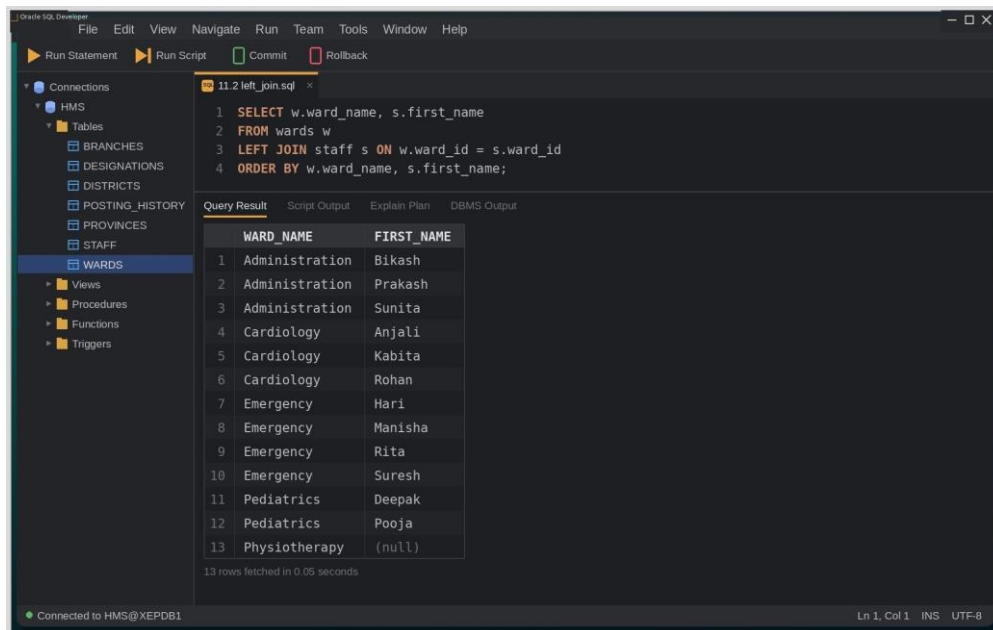
Query:

```
SELECT e.first_name AS staff_member, s.first_name AS supervisor  
FROM staff e  
JOIN staff s ON e.supervisor_id = s.staff_id  
WHERE e.ward_id = 50  
ORDER BY staff_member;
```

Output:

STAFF_MEMBER	SUPERVISOR
Hari	Suresh
Manisha	Suresh
Rita	Suresh
Suresh	Prakash

Screenshot (Execution Output):



11.2 left_join.sql

```
1 SELECT w.ward_name, s.first_name
2 FROM wards w
3 LEFT JOIN staff s ON w.ward_id = s.ward_id
4 ORDER BY w.ward_name, s.first_name;
```

	WARD_NAME	FIRST_NAME
1	Administration	Bikash
2	Administration	Prakash
3	Administration	Sunita
4	Cardiology	Anjali
5	Cardiology	Kabita
6	Cardiology	Rohan
7	Emergency	Hari
8	Emergency	Manisha
9	Emergency	Rita
10	Emergency	Suresh
11	Pediatrics	Deepak
12	Pediatrics	Pooja
13	Physiotherapy	(null)

13 rows fetched in 0.05 seconds

Connected to HMS@XEPDB1 Ln 1, Col 1 INS UTF-8

Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

The difference between the two join types was obvious once the results were placed side by side: the inner join returned twelve rows, while the left join returned thirteen because the empty Physiotherapy ward was kept with a NULL name. The self join needed two aliases for the same STAFF table — one acting as the employee and one as the supervisor — and once I read E and S as two separate copies of the table, the query was easy to follow.

Choosing between an inner and an outer join depends on whether unmatched rows carry meaning for the report; here, an empty ward is exactly the kind of information the hospital would want to see.

Lab-12: Implement database views by creating, modifying, querying, and dropping views for efficient data access.

Group Type: Individual

Objective:

To create, query, replace and drop database views that simplify access to frequently used data.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

A view is a stored query that can be referenced like a table but holds no data of its own; each time it is queried, Oracle runs the underlying SELECT against the current base tables. Views hide complex joins, can expose only selected columns or rows for security, and keep application queries stable even when the base tables are reorganised.

Query / Program and Output:

CREATE VIEW

Syntax:

```
CREATE [OR REPLACE] VIEW view_name AS SELECT ...;
```

Query:

```
CREATE OR REPLACE VIEW ward_payroll_vw AS
SELECT w.ward_name, COUNT(s.staff_id) AS headcount,
SUM(s.salary) AS total_salary
FROM wards w
LEFT JOIN staff s ON w.ward_id = s.ward_id
GROUP BY w.ward_name;
```

Querying the view

Query:

```
SELECT * FROM ward_payroll_vw ORDER BY total_salary DESC NULLS LAST;
```

Output:

WARD_NAME	HEADCOUNT	TOTAL_SALARY
Administration	3	605000
Cardiology	3	303000
Emergency	4	242000
Pediatrics	2	196000
Physiotherapy	0	NULL

Modifying and dropping

Query:

```
CREATE OR REPLACE VIEW ward_payroll_vw AS
SELECT w.ward_name, COUNT(s.staff_id) AS headcount
FROM wards w LEFT JOIN staff s
ON w.ward_id = s.ward_id
GROUP BY w.ward_name;

DROP VIEW ward_payroll_vw;
```

Screenshot (Execution Output):

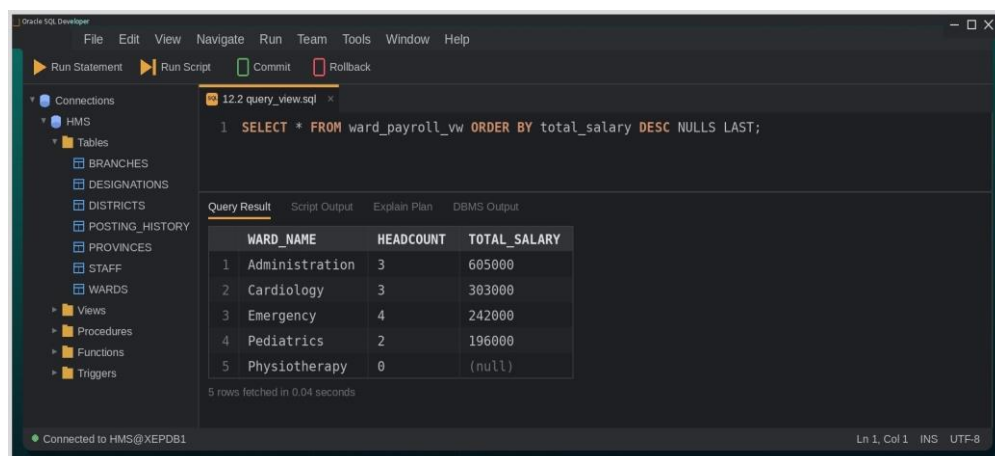


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

Querying `WARD_PAYROLL_VW` felt exactly like reading a table, even though every `SELECT` re-ran the join and aggregation behind it. The empty Physiotherapy ward showed a headcount of 0 but a `NULL` total salary, because `COUNT` of a column ignores `NULL`s while `SUM` over no values is `NULL`; `NULLS LAST` kept that row at the bottom. I changed the definition with `CREATE OR REPLACE` rather than dropping and recreating the view, which keeps any privileges already granted on it.

Views give users a simple, pre-joined picture of the data without exposing the base tables behind it.

Lab-13: Implement SQL subqueries, including single-row, multiple-row, multiple-column, and correlated subqueries for data retrieval and manipulation.

Group Type: Individual

Objective:

To write single-row, multiple-row, multiple-column and correlated subqueries for retrieving data from the hospital database.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

A subquery is a query placed inside another SQL statement whose result is used by the outer statement. A single-row subquery returns one value and is compared with =, <, > and similar operators; a multiple-row subquery returns a list and needs IN, ANY or ALL; a multiple-column subquery returns pairs of values that are compared together; and a correlated subquery refers to a column of the outer query, so Oracle evaluates it again for every outer row.

Query / Program and Output:

Single-row subquery

Query:

```
SELECT first_name, salary FROM staff  
WHERE salary > (SELECT AVG(salary) FROM staff);
```

Multiple-row subquery

Query:

```
SELECT first_name, ward_id FROM staff  
WHERE ward_id IN  
(SELECT ward_id FROM wards WHERE branch_id = 1200);
```

Multiple-column subquery

Query:

```
SELECT staff_id, desig_code, salary FROM staff
```

```
WHERE (desig_code, salary) IN  
(SELECT desig_code, MIN(salary) FROM staff GROUP BY desig_code);
```

Correlated subquery

Query:

```
SELECT s.first_name, s.salary, s.ward_id  
FROM staff s  
WHERE s.salary > (  
SELECT AVG(salary) FROM staff s2  
WHERE s2.ward_id = s.ward_id  
);
```

Output:

FIRST_NAME	SALARY	WARD_ID
Prakash	250000	10
Anjali	160000	20
Deepak	150000	40
Suresh	98000	50
Hari	62000	50

Screenshot (Execution Output):

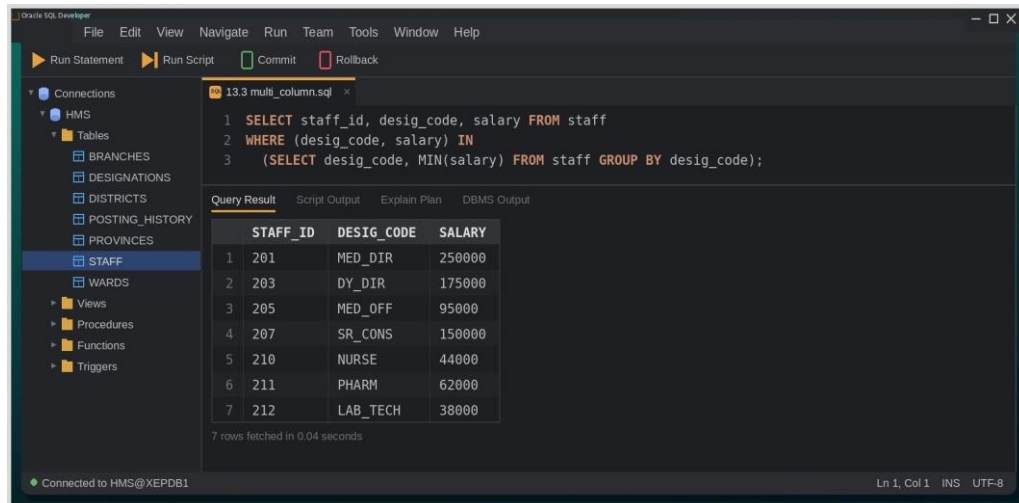


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

Tracing the correlated subquery by hand was the clearest way to understand it: for every staff row Oracle recomputes the average of that person's own ward, so Suresh (98000) and Hari (62000) both qualify against Emergency's low average of 60500, while Rohan's 95000 does not beat Cardiology's 101000. The multiple-column subquery in 13.3 answered 'who is the lowest-paid person in each designation' in a single statement, where otherwise a join with a grouped inline view would have been required.

Subqueries allow a question to be answered in stages inside one statement, keeping the logic readable and avoiding temporary tables.

Lab-14: Write a PL/SQL program to illustrate the Conditional Statements in PL/SQL.

Group Type: Individual

Objective:

To write PL/SQL blocks that use the conditional control statements IF-THEN-ELSIF and CASE.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

PL/SQL adds procedural logic to SQL. Its conditional statements IF-THEN, IF-THEN-ELSE, IF-THEN-ELSIF and CASE (simple and searched) decide which statements run based on Boolean conditions, and the conditions are tested from top to bottom until the first one that evaluates to TRUE.

Query / Program and Output:

IF-THEN-ELSIF

Query:

```
DECLARE

v_salary staff.salary%TYPE;

BEGIN

SELECT salary INTO v_salary FROM staff WHERE staff_id = 205;


IF v_salary >= 150000 THEN
DBMS_OUTPUT.PUT_LINE('Consultant grade');
ELSIF v_salary >= 80000 THEN
DBMS_OUTPUT.PUT_LINE('Officer grade');
ELSE
DBMS_OUTPUT.PUT_LINE('Support grade');
END IF;

END;

/
```

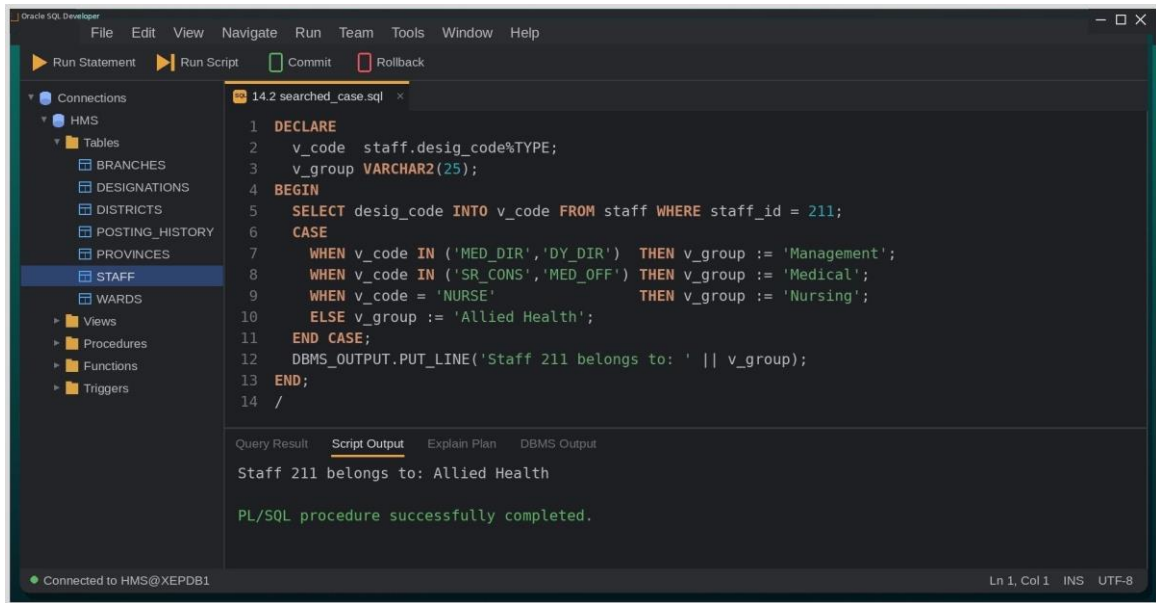
Output:

OUTPUT
Officer grade

CASE statement**Query:**

```
DECLARE
v_code staff.desig_code%TYPE;
v_group VARCHAR2(25);
BEGIN
SELECT desig_code INTO v_code FROM staff WHERE staff_id = 211;
CASE
WHEN v_code IN ('MED_DIR','DY_DIR') THEN v_group := 'Management';
WHEN v_code IN ('SR_CONS','MED_OFF') THEN v_group := 'Medical';
WHEN v_code = 'NURSE' THEN v_group := 'Nursing';
ELSE v_group := 'Allied Health';
END CASE;
DBMS_OUTPUT.PUT_LINE('Staff 211 belongs to: ' || v_group);
END;
/
```

Screenshot (Execution Output):



```
1 DECLARE
2   v_code  staff.desig_code%TYPE;
3   v_group VARCHAR2(25);
4 BEGIN
5   SELECT desig_code INTO v_code FROM staff WHERE staff_id = 211;
6   CASE
7     WHEN v_code IN ('MED_DIR','DY_DIR') THEN v_group := 'Management';
8     WHEN v_code IN ('SR_CONS','MED_OFF') THEN v_group := 'Medical';
9     WHEN v_code = 'NURSE' THEN v_group := 'Nursing';
10    ELSE v_group := 'Allied Health';
11  END CASE;
12  DBMS_OUTPUT.PUT_LINE('Staff 211 belongs to: ' || v_group);
13 END;
14 /
```

Query Result **Script Output** Explain Plan DBMS Output

Staff 211 belongs to: Allied Health

PL/SQL procedure successfully completed.

Connected to HMS@XEPDB1 Ln 1, Col 1 INS UTF-8

Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

Order matters in an IF-ELSIF ladder. In my first attempt the ≥ 80000 test came before the ≥ 150000 test, so Anjali and Deepak were wrongly reported as 'Officer grade' — the broader condition matched first and the correct branch was never reached. Anchoring V_SALARY with %TYPE meant the variable automatically follows the column's datatype. The searched CASE in 14.2 was tidier than nested IFs for mapping designation codes to staff groups.

Once the conditions are arranged from most specific to most general, PL/SQL branching behaves predictably.

Lab-15: Write a PL/SQL program to illustrate the Iterative Statements in PL/SQL.

Group Type: Individual

Objective:

To write PL/SQL blocks that use the iterative statements LOOP, WHILE LOOP and FOR LOOP.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

PL/SQL offers three looping constructs: the basic LOOP, which repeats until an EXIT or EXIT WHEN condition is met; the WHILE loop, which checks its condition before every iteration; and the FOR loop, which runs over a fixed numeric range or over the rows returned by a query (cursor FOR loop).

Query / Program and Output:

Basic LOOP

Query:

```
DECLARE
v_i NUMBER := 1;
BEGIN
LOOP
DBMS_OUTPUT.PUT_LINE('7 x ' || v_i || '=' || 7 * v_i);
v_i := v_i + 1;
EXIT WHEN v_i > 5;
END LOOP;
END;
/
```

WHILE loop

Query:

```
DECLARE
```

```

v_beds NUMBER := 3;
v_year NUMBER := 0;
BEGIN
WHILE v_beds < 50 LOOP
v_beds := v_beds * 2;
v_year := v_year + 1;
END LOOP;
DBMS_OUTPUT.PUT_LINE('ICU beds after ' || v_year || ' years: ' || v_beds);
END;
/

```

FOR loop over query results

Query:

```

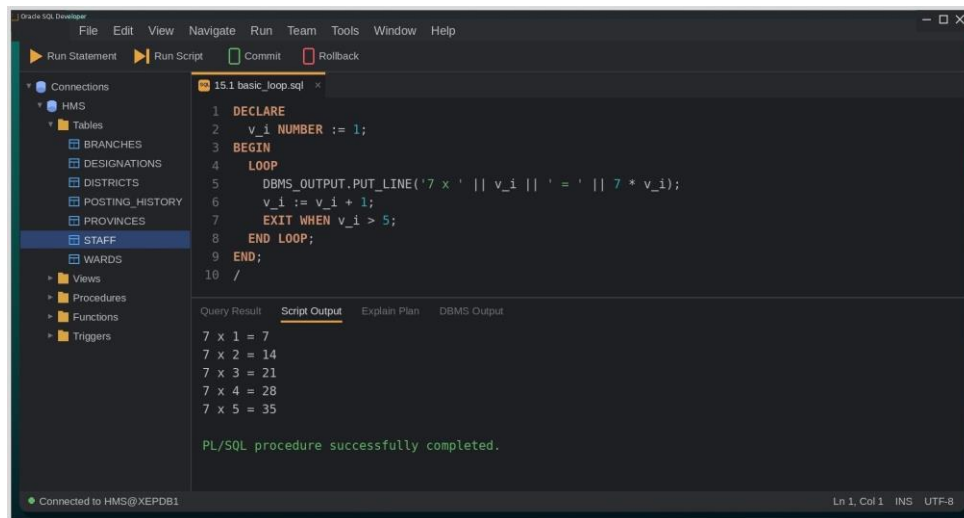
BEGIN
FOR r IN (SELECT first_name, salary FROM staff WHERE ward_id = 40) LOOP
DBMS_OUTPUT.PUT_LINE(r.first_name || ' draws ' || r.salary);
END LOOP;
END;
/

```

Output:

OUTPUT
Deepak draws 150000
Pooja draws 46000

Screenshot (Execution Output):



```
1 DECLARE
2   v_i NUMBER := 1;
3 BEGIN
4   LOOP
5     DBMS_OUTPUT.PUT_LINE('7 x ' || v_i || ' = ' || 7 * v_i);
6     v_i := v_i + 1;
7     EXIT WHEN v_i > 5;
8   END LOOP;
9 END;
10 /
```

Query Result **Script Output** Explain Plan DBMS Output

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35

PL/SQL procedure successfully completed.
```

Connected to HMS@XEPDB1 Ln 1, Col 1 INS UTF-8

Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

The cursor FOR loop needed the least code, because Oracle opens, fetches from and closes the cursor on its own. The WHILE loop taught me to always update the controlling variable: in one test I left out the V_BEDS line, the condition never became false and the session had to be cancelled. The basic LOOP suited the multiplication table, since its exit test naturally sits after the print (the WHILE version printed 'ICU beds after 5 years: 96').

For processing query rows the cursor FOR loop is the most practical default; the other two suit counter-driven or condition-driven repetition.

Lab-16: Write a PL/SQL program to illustrate the Sequential Control in PL/SQL.

Group Type: Individual

Objective:

To write PL/SQL blocks that use the sequential control statements GOTO, CONTINUE and NULL.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Sequential control statements change the normal top-to-bottom flow of a PL/SQL block. GOTO jumps unconditionally to a labelled statement in the same or an enclosing block; CONTINUE skips the rest of the current loop iteration and starts the next; EXIT leaves the loop entirely; and NULL does nothing, acting as a placeholder where the syntax requires a statement.

Query / Program and Output:

GOTO

Query:

```
DECLARE
v_token NUMBER := 1;
BEGIN
<<issue_token>>
IF v_token > 3 THEN
GOTO counter_closed;
END IF;
DBMS_OUTPUT.PUT_LINE('OPD token issued: ' || v_token);
v_token := v_token + 1;
GOTO issue_token;
<<counter_closed>>
DBMS_OUTPUT.PUT_LINE('Counter closed');
END;
/
```

CONTINUE

Query:

```
BEGIN
FOR i IN 1..10 LOOP
IF MOD(i, 3) <> 0 THEN
CONTINUE;
END IF;
DBMS_OUTPUT.PUT_LINE('Multiple of 3: ' || i);
END LOOP;
END;
/
```

Output:

OUTPUT
Multiple of 3: 3
Multiple of 3: 6
Multiple of 3: 9

NULL statement

Query:

```
DECLARE
v_bonus staff.bonus_pct%TYPE;
BEGIN
SELECT bonus_pct INTO v_bonus FROM staff WHERE staff_id = 206;
IF v_bonus IS NULL THEN
NULL; -- bonus rule to be added in the next release
ELSE
DBMS_OUTPUT.PUT_LINE('Bonus: ' || v_bonus * 100 || '%');
```

```
END IF;  
END;  
/
```

Screenshot (Execution Output):

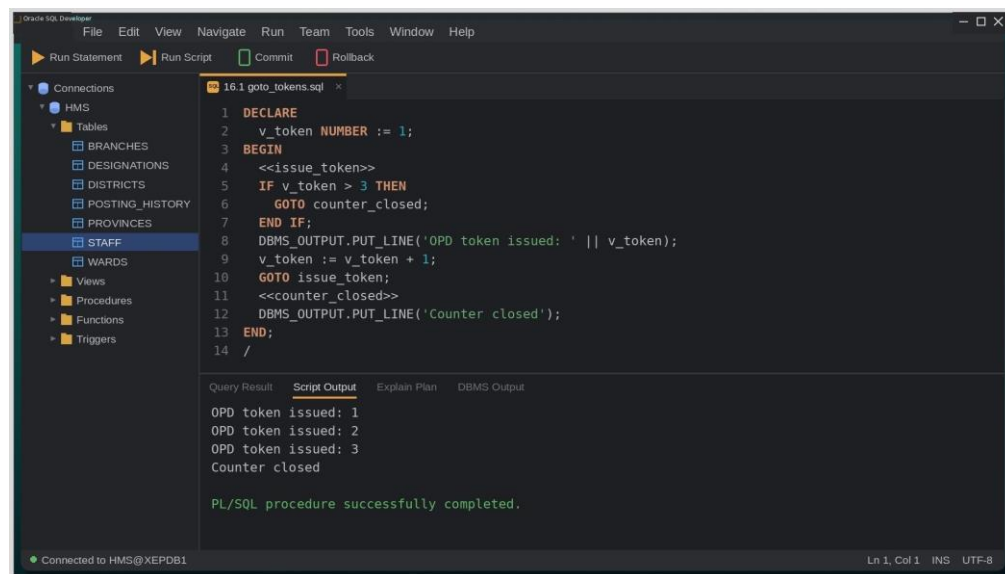


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

The GOTO version works but jumping back to `ISSUE_TOKEN` is really a loop written the hard way and is harder to read than an equivalent FOR loop. Oracle also rejected, at compile time, a GOTO that tried to jump into an IF block from outside it (PLS-00375). `CONTINUE` was far more natural for printing only the multiples of three, and the `NULL` statement kept the IF structure valid for staff 206, whose `BONUS_PCT` is `NULL`, without printing anything.

These statements are worth knowing, but in real programs structured loops with `CONTINUE` and `EXIT` are almost always clearer than GOTO.

Lab-17: Write a PL/SQL program to demonstrate the use of an Implicit Cursor.

Group Type: Individual

Objective:

To write PL/SQL blocks that use implicit cursor attributes to check the outcome of SQL statements.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Whenever PL/SQL runs a DML statement or a SELECT INTO, Oracle manages an implicit cursor named SQL behind the scenes. Its attributes describe the most recent statement: SQL%FOUND and SQL%NOTFOUND tell whether any row was affected, SQL%ROWCOUNT gives the number of rows, and SQL%ISOPEN is always FALSE because Oracle closes the cursor automatically.

Query / Program and Output:

SQL%ROWCOUNT / SQL%FOUND

Query:

```
BEGIN
UPDATE staff SET bonus_pct = 0.05
WHERE desig_code = 'NURSE' AND bonus_pct IS NULL;

IF SQL%FOUND THEN
DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' nurse record(s) updated. ');
ELSE
DBMS_OUTPUT.PUT_LINE('No nurse needed an update. ');
END IF;
ROLLBACK;
END;
/
```

Output:

OUTPUT
3 nurse record(s) updated.

Implicit cursor with SELECT INTO

Query:

```
BEGIN
DELETE FROM staff WHERE ward_id = 80;
IF SQL%NOTFOUND THEN
DBMS_OUTPUT.PUT_LINE('Physiotherapy ward has no staff to remove. ');
END IF;
END;
/
```

Screenshot (Execution Output):

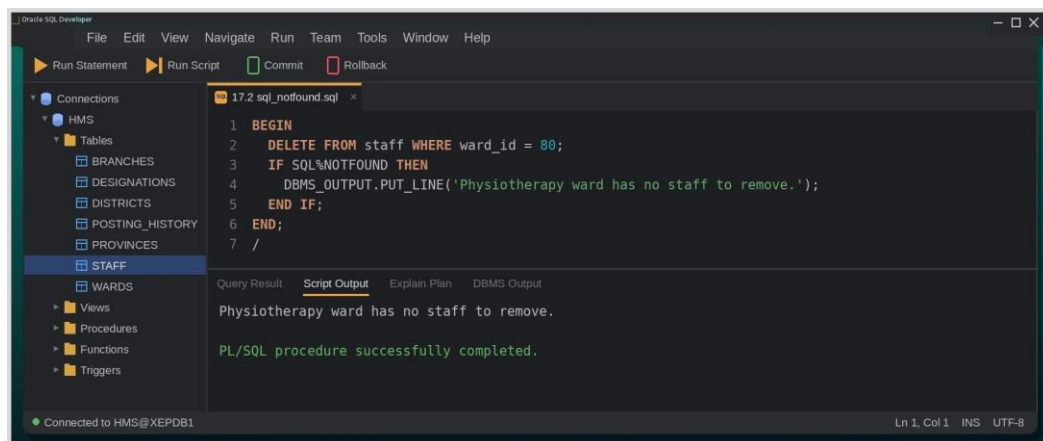


Fig. Execution output of the statements above, captured from the client tool.

17.5. Discussion & Conclusion:

SQL%ROWCOUNT worked as an instant sanity check: I expected three nurses, got 3 back, and knew the WHERE clause was right before the ROLLBACK undid the change. In 17.2 the DELETE on the empty Physiotherapy ward touched no rows, so SQL%NOTFOUND was TRUE. I also realized that a SELECT COUNT(*) INTO would

not work for this kind of test an aggregate always returns exactly one row, so SQL%NOTFOUND would stay FALSE even when the count is zero.

The attributes have to be read immediately after the statement they describe, because the next SQL statement overwrites them.

Lab-18: Write a PL/SQL program to demonstrate the use of an Implicit Cursor.

Group Type: Individual

Objective:

To write PL/SQL blocks that process query results row by row using explicit cursors, including a parameterized cursor.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

An explicit cursor is a named pointer to the result set of a multi-row SELECT. The programmer controls its life cycle DECLARE, OPEN, FETCH (usually inside a loop) and CLOSE and uses the attributes %FOUND, %NOTFOUND, %ROWCOUNT and %ISOPEN to decide when to stop. Cursors can also accept parameters, so a single declaration can serve many different filter values.

Query / Program and Output:

Explicit cursor with a basic loop

Query:

```
DECLARE
CURSOR er_cur IS
SELECT first_name, salary FROM staff
WHERE ward_id = 50 ORDER BY staff_id;
v_name staff.first_name%TYPE;
v_sal staff.salary%TYPE;
BEGIN
OPEN er_cur;
LOOP
FETCH er_cur INTO v_name, v_sal;
EXIT WHEN er_cur%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(er_cur%ROWCOUNT || ' ' || v_name || ' ' || v_sal);
END LOOP;
CLOSE er_cur;
```

```
END;
```

```
/
```

Output:

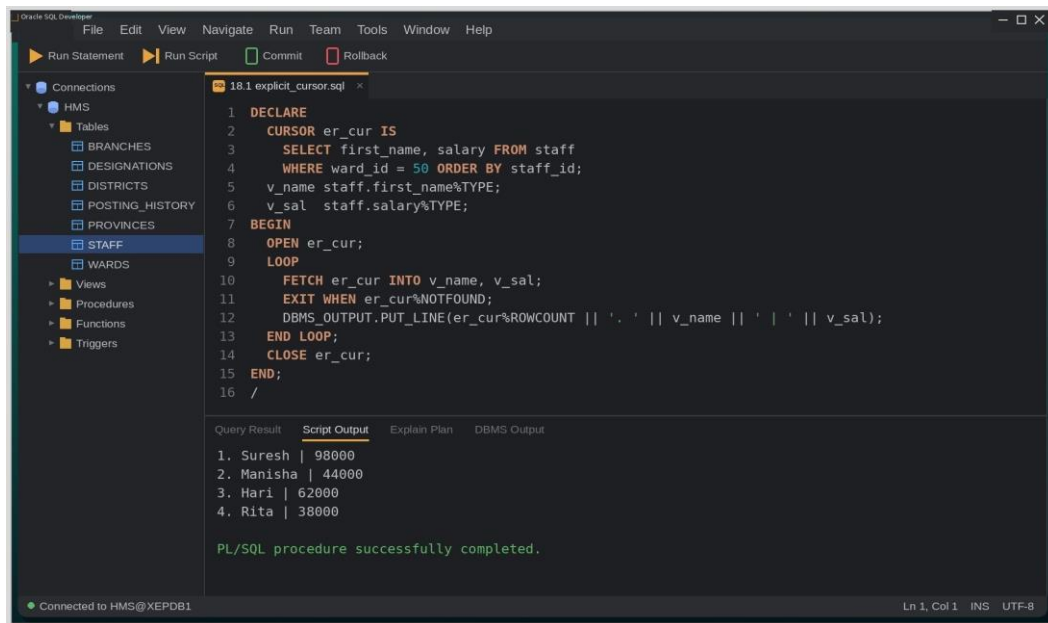
OUTPUT
1. Suresh 98000
2. Manisha 44000
3. Hari 62000
4. Rita 38000

Explicit cursor with parameters

Query:

```
DECLARE
CURSOR desig_cur (p_code VARCHAR2) IS
SELECT first_name FROM staff
WHERE desig_code = p_code ORDER BY join_date;
v_name staff.first_name%TYPE;
BEGIN
OPEN desig_cur('NURSE');
FETCH desig_cur INTO v_name;
DBMS_OUTPUT.PUT_LINE('Longest-serving nurse: ' || v_name);
CLOSE desig_cur;
END;
/
```

Screenshot (Execution Output):



```
1 DECLARE
2   CURSOR er_cur IS
3     SELECT first_name, salary FROM staff
4     WHERE ward_id = 50 ORDER BY staff_id;
5   v_name staff.first_name%TYPE;
6   v_sal  staff.salary%TYPE;
7 BEGIN
8   OPEN er_cur;
9   LOOP
10    FETCH er_cur INTO v_name, v_sal;
11    EXIT WHEN er_cur%NOTFOUND;
12    DBMS_OUTPUT.PUT_LINE(er_cur%ROWCOUNT || ' | ' || v_name || ' | ' || v_sal);
13  END LOOP;
14  CLOSE er_cur;
15 END;
16 /
```

Query Result **Script Output** Explain Plan DBMS Output

```
1. Suresh | 98000
2. Manisha | 44000
3. Hari | 62000
4. Rita | 38000

PL/SQL procedure successfully completed.
```

Connected to HMS@XEPDB1 Ln 1, Col 1 INS UTF-8

Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

Printing %ROWCOUNT inside the loop gave a running serial number for free, since it counts the rows fetched so far. The position of EXIT WHEN mattered: when I moved it below the PUT_LINE, the last row (Rita) was printed twice, because the final, failed fetch leaves the previous values in the variables. The parameterized cursor in answered the same question for any designation just by changing the argument, and returned Pooja as the longest-serving nurse.

Explicit cursors need more code than a FOR loop, but that control becomes necessary once the row-by-row logic grows beyond simple printing.

Lab-19: Write a PL/SQL program to illustrate Exception Handling using Predefined Exceptions.

Group Type: Individual

Objective:

To handle run-time errors in PL/SQL using Oracle's predefined exceptions.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

When a PL/SQL block meets a run-time error, Oracle raises an exception and passes control to the block's EXCEPTION section. The most common errors have predefined names NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, DUP_VAL_ON_INDEX, VALUE_ERROR so they can be handled by name in WHEN clauses, while WHEN OTHERS catches anything not listed.

Query / Program and Output:

NO_DATA_FOUND

Query:

```
DECLARE
v_name staff.first_name%TYPE;
BEGIN
SELECT first_name INTO v_name FROM staff WHERE staff_id = 999;
EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('Staff ID 999 does not exist.');
```

END;

/

Output:

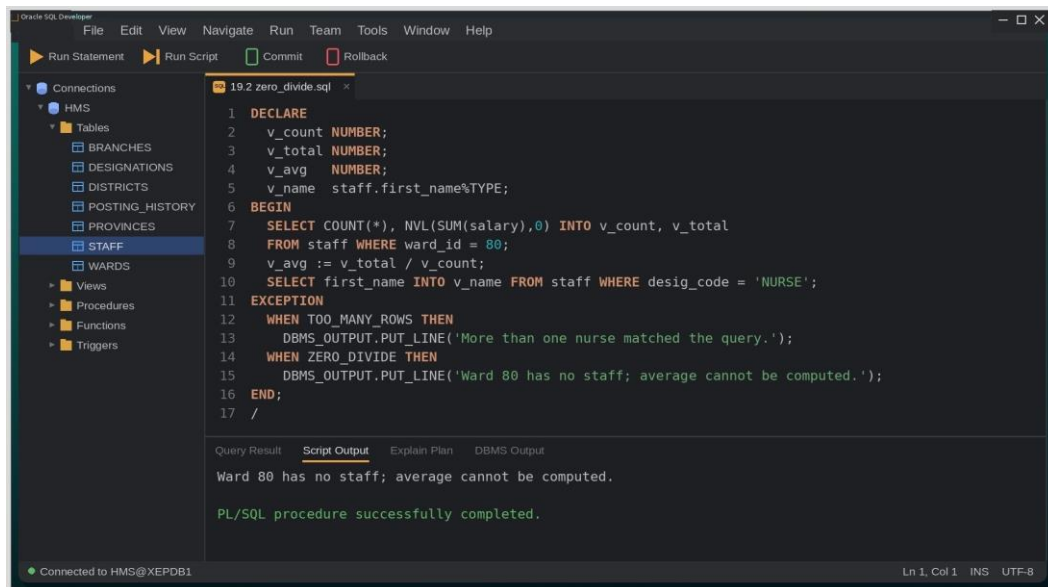
OUTPUT
Staff ID 999 does not exist.

TOO_MANY_ROWS and ZERO_DIVIDE

Query:

```
DECLARE
v_count NUMBER;
v_total NUMBER;
v_avg  NUMBER;
v_name staff.first_name%TYPE;
BEGIN
SELECT COUNT(*), NVL(SUM(salary),0) INTO v_count, v_total
FROM staff WHERE ward_id = 80;
v_avg := v_total / v_count;
SELECT first_name INTO v_name FROM staff WHERE desig_code = 'NURSE';
EXCEPTION
WHEN TOO_MANY_ROWS THEN
DBMS_OUTPUT.PUT_LINE('More than one nurse matched the query. ');
WHEN ZERO_DIVIDE THEN
DBMS_OUTPUT.PUT_LINE('Ward 80 has no staff; average cannot be computed. ');
END;
/
```

Screenshot (Execution Output):



```
1 DECLARE
2   v_count NUMBER;
3   v_total NUMBER;
4   v_avg   NUMBER;
5   v_name  staff.first_name%TYPE;
6 BEGIN
7   SELECT COUNT(*), NVL(SUM(salary),0) INTO v_count, v_total
8   FROM staff WHERE ward_id = 80;
9   v_avg := v_total / v_count;
10  SELECT first_name INTO v_name FROM staff WHERE design_code = 'NURSE';
11 EXCEPTION
12  WHEN TOO_MANY_ROWS THEN
13    DBMS_OUTPUT.PUT_LINE('More than one nurse matched the query.');

Query Result   Script Output   Explain Plan   DBMS Output



Ward 80 has no staff; average cannot be computed.



PL/SQL procedure successfully completed.



Connected to HMS@XEPDB1



Ln 1, Col 1: INS UTF-8


```

Fig. Execution output of the statements above, captured from the client tool.

19.5. Discussion & Conclusion:

In 19.2 two errors were waiting to happen, but only ZERO_DIVIDE was reported: the division by Physiotherapy's zero head count came first, and once an exception is raised the rest of the executable section is abandoned, so the TOO_MANY_ROWS query never ran. When I swapped the two statements, the TOO_MANY_ROWS handler fired instead, since three nurses match that query. The order of the WHEN clauses, on the other hand, made no difference at all.

Predefined exceptions let the common failures be handled by readable names instead of memorized Oracle error numbers.

Lab-20: Write a PL/SQL program to illustrate Exception Handling using User-Defined Exceptions.

Group Type: Individual

Objective:

To declare, raise and handle user-defined exceptions in PL/SQL, and to return custom errors with RAISE_APPLICATION_ERROR.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Business rules such as 'a bonus must not exceed 25%' are not errors Oracle knows about. PL/SQL lets the programmer declare a variable of type EXCEPTION, raise it with RAISE when the rule is broken and handle it like any other exception. RAISE_APPLICATION_ERROR goes a step further by sending an error number between -20000 and -20999, together with a custom message, back to the calling program.

Query / Program and Output:

Declaring and raising a custom exception

Query:

```
DECLARE
e_no_bonus EXCEPTION;
v_name staff.first_name%TYPE;
v_bonus staff.bonus_pct%TYPE;
BEGIN
SELECT first_name, bonus_pct INTO v_name, v_bonus
FROM staff WHERE staff_id = 206;

IF v_bonus IS NULL THEN
RAISE e_no_bonus;
END IF;
EXCEPTION
WHEN e_no_bonus THEN
```

```
DBMS_OUTPUT.PUT_LINE(v_name || ' is not eligible for a bonus this cycle.');
```

```
END;
```

```
/
```

Output:

OUTPUT
Kabita is not eligible for a bonus this cycle.

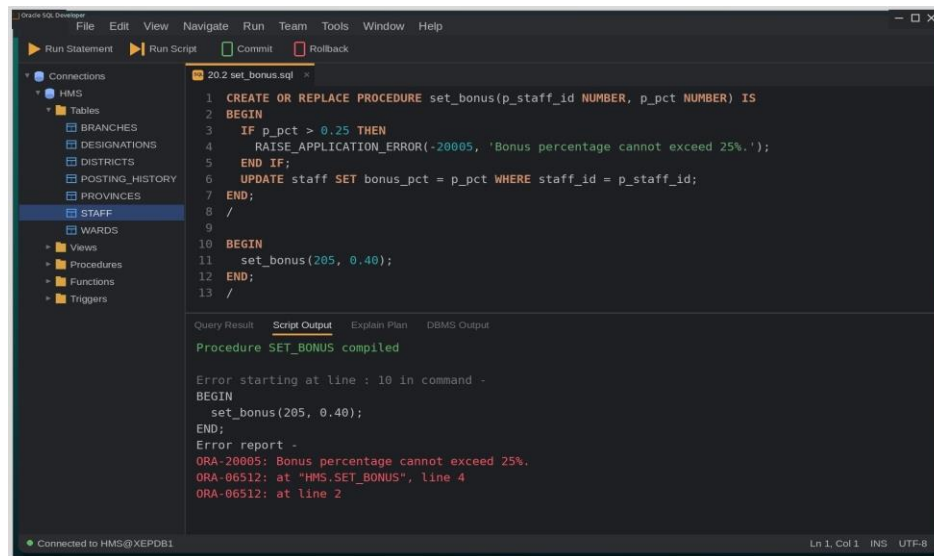
RAISE_APPLICATION_ERROR

Query:

```
CREATE OR REPLACE PROCEDURE set_bonus(p_staff_id NUMBER, p_pct
NUMBER) IS
BEGIN
IF p_pct > 0.25 THEN
RAISE_APPLICATION_ERROR(-20005, 'Bonus percentage cannot exceed 25%.');
END IF;
UPDATE staff SET bonus_pct = p_pct WHERE staff_id = p_staff_id;
END;
/

BEGIN
set_bonus(205, 0.40);
END;
/
```

Screenshot (Execution Output):



```
1 CREATE OR REPLACE PROCEDURE set_bonus(p_staff_id NUMBER, p_pct NUMBER) IS
2 BEGIN
3 IF p_pct > 0.25 THEN
4 RAISE_APPLICATION_ERROR(-20005, 'Bonus percentage cannot exceed 25%.');
5 END IF;
6 UPDATE staff SET bonus_pct = p_pct WHERE staff_id = p_staff_id;
7 END;
8 /
9
10 BEGIN
11 set_bonus(205, 0.40);
12 END;
13 /
```

Query Result Script Output Explain Plan DBMS Output

Procedure SET_BONUS compiled

Error starting at line : 10 in command -
BEGIN
set_bonus(205, 0.40);
END;
Error report -
ORA-20005: Bonus percentage cannot exceed 25%.
ORA-06512: at "HMS.SET_BONUS", line 4
ORA-06512: at line 2

Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

The two techniques suit different situations. E_NO_BONUS was handled inside the same block, so it only produced a friendly message. RAISE_APPLICATION_ERROR in SET_BONUS instead stopped the procedure and returned ORA-20005 with my own text to the caller, followed by ORA-06512 lines pointing to the exact line the behavior a real front end needs in order to display a validation message.

User-defined exceptions turn hospital policies into rules that the database enforces, rather than guidelines the application merely hopes to follow.

Lab-21: Create PL/SQL stored procedures using IN, OUT, and IN OUT parameters to illustrate modularity.

Group Type: Individual

Objective:

To create and execute stored procedures that use the IN, OUT and IN OUT parameter modes.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

A stored procedure is a named PL/SQL block that is compiled and saved in the database, so any authorized program can call it. Each parameter has one of three modes: IN passes a read-only value into the procedure, OUT returns a value to the caller, and IN OUT passes an initial value in and hands a possibly modified value back.

Query / Program and Output:

IN and OUT parameters

Query:

```
CREATE OR REPLACE PROCEDURE ward_stats (  
  p_ward_id IN wards.ward_id%TYPE,  
  p_count   OUT NUMBER,  
  p_payroll OUT NUMBER  
) IS  
BEGIN  
  SELECT COUNT(*), NVL(SUM(salary),0)  
  INTO p_count, p_payroll  
  FROM staff  
  WHERE ward_id = p_ward_id;  
END;  
/
```

Calling the procedure

Query:

```
DECLARE  
  
v_count NUMBER;  
v_payroll NUMBER;  
  
BEGIN  
  
ward_stats(50, v_count, v_payroll);  
  
DBMS_OUTPUT.PUT_LINE('Staff: ' || v_count || ', Monthly payroll: ' || v_payroll);  
  
END;  
  
/
```

Output:

OUTPUT
Staff: 4, Monthly payroll: 242000

IN OUT parameter

Query:

```
CREATE OR REPLACE PROCEDURE add_night_allowance (p_pay IN OUT  
NUMBER, p_nights IN NUMBER) IS  
  
BEGIN  
  
p_pay := p_pay + (p_nights * 1200);  
  
END;  
  
/  
  
DECLARE  
  
v_pay NUMBER := 46000;  
  
BEGIN  
  
add_night_allowance(v_pay, 5);
```

```

DBMS_OUTPUT.PUT_LINE('Pay with night allowance: ' || v_pay);

END;

/

```

Screenshot (Execution Output):

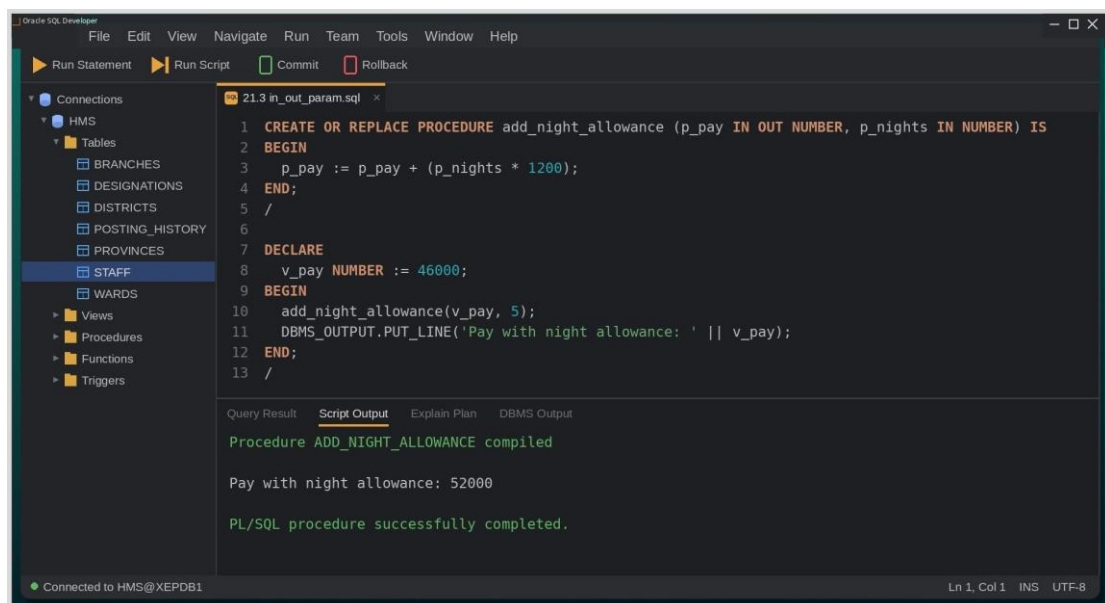


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

WARD_STATS showed why OUT parameters are useful: one call returned both the head count and the payroll of the Emergency ward, whereas a function would have needed a record type to return two values. The IN OUT example made the mode tangible — V_PAY entered ADD_NIGHT_ALLOWANCE as 46000 and came back as 52000 after five nights at 1200 each. An attempt to assign a value to P_NIGHTS inside the procedure failed with PLS-00363, confirming that IN parameters are read-only.

Parameter modes define a clear contract between a procedure and its caller, which is what makes a procedure safe to reuse.

Lab-22: Write a PL/SQL procedures to illustrate the CRUD operations with the transaction control like commit, rollback in the database using stored procedure.

Group Type: Individual

Objective:

To implement CRUD operations through PL/SQL stored procedures that control their own transactions with COMMIT and ROLLBACK.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Placing the insert, read, update and delete logic inside stored procedures gives every application the same tested entry point to the data. Each procedure can check the result of its own work and decide whether to COMMIT or ROLLBACK, so a half-finished change is never left behind, whichever program made the call.

Query / Program and Output:

CREATE + transaction control

Query:

```
CREATE OR REPLACE PROCEDURE hire_staff (  
p_id NUMBER, p_first VARCHAR2, p_last VARCHAR2,  
p_email VARCHAR2, p_desig VARCHAR2, p_salary NUMBER, p_ward NUMBER  
) IS  
BEGIN  
INSERT INTO staff (staff_id, first_name, last_name, email,  
join_date, desig_code, salary, ward_id)  
VALUES (p_id, p_first, p_last, p_email, SYSDATE, p_desig, p_salary, p_ward);  
COMMIT;  
EXCEPTION  
WHEN OTHERS THEN  
ROLLBACK;  
RAISE;  
END;
```

/

UPDATE and DELETE procedures

Query:

```
CREATE OR REPLACE PROCEDURE revise_salary (p_id NUMBER, p_salary  
NUMBER) IS
```

```
BEGIN
```

```
UPDATE staff SET salary = p_salary WHERE staff_id = p_id;
```

```
IF SQL%ROWCOUNT = 0 THEN
```

```
ROLLBACK;
```

```
RAISE_APPLICATION_ERROR(-20012, 'No staff member with ID ' || p_id);
```

```
END IF;
```

```
COMMIT;
```

```
END;
```

```
/
```

```
CREATE OR REPLACE PROCEDURE remove_staff (p_id NUMBER) IS
```

```
BEGIN
```

```
DELETE FROM staff WHERE staff_id = p_id;
```

```
COMMIT;
```

```
END;
```

```
/
```

Exercising the procedures

Query:

```
EXEC hire_staff(320,'Navnit','Dangi','NDANGI','PHARM',58000,50);
```

```
EXEC revise_salary(320, 61000);
```

```
EXEC revise_salary(999, 61000);
```

```
EXEC remove_staff(320);
```

Screenshot (Execution Output):

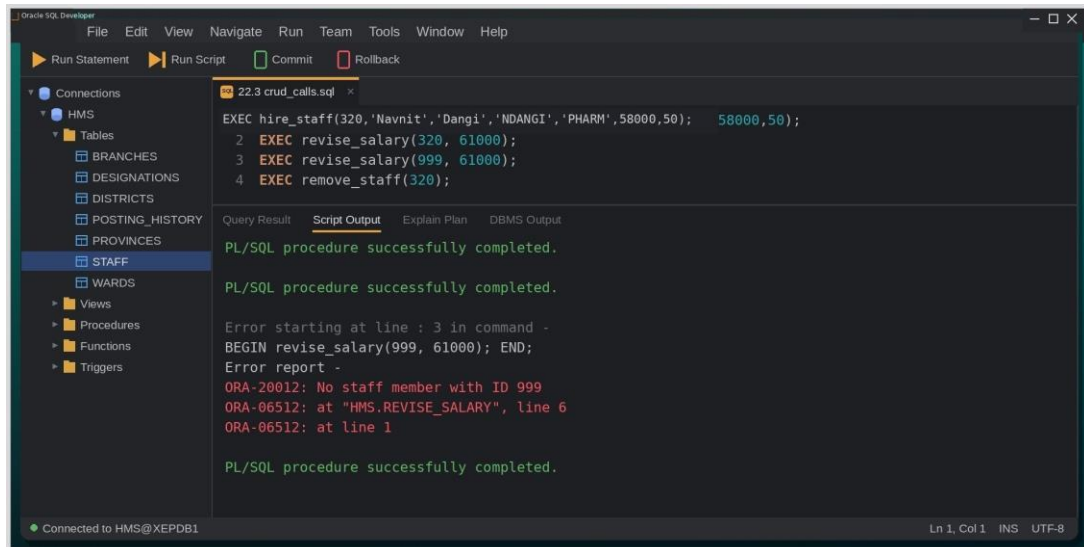


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

Calling REVISE_SALARY with the non-existent ID 999 deliberately tested the failure path: SQL%ROWCOUNT was 0, so the procedure rolled back and returned ORA-20012 with the ID in the message instead of quietly reporting success. In HIRE_STAFF, the bare RAISE after ROLLBACK turned out to be important my first version only printed a message in WHEN OTHERS, which hid the real ORA-00001 error when I inserted a duplicate ID.

Keeping COMMIT and ROLLBACK inside the procedures gives every caller the same all-or-nothing behavior without having to remember it.

Lab-23: Create user-defined PL/SQL functions of your choice for the given database and demonstrate their use in SQL queries and PL/SQL blocks.

Group Type: Individual

Objective:

To create user-defined PL/SQL functions and call them from SQL statements as well as from PL/SQL blocks.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

A stored function is a named PL/SQL subprogram that must return exactly one value through the RETURN statement. Because it produces a value, a function can be used wherever an expression is allowed — in a SELECT list, a WHERE clause or a PL/SQL assignment as long as it performs no DML when called from a query.

Query / Program and Output:

Function definition

Query:

```
CREATE OR REPLACE FUNCTION yearly_package (p_staff_id NUMBER)
RETURN NUMBER IS
v_salary staff.salary%TYPE;
v_bonus  staff.bonus_pct%TYPE;
BEGIN
SELECT salary, NVL(bonus_pct, 0) INTO v_salary, v_bonus
FROM staff WHERE staff_id = p_staff_id;
RETURN (v_salary * 12) * (1 + v_bonus);
EXCEPTION
WHEN NO_DATA_FOUND THEN
RETURN NULL;
END;
/
```

Using the function in SQL

Query:

```
SELECT first_name, salary, yearly_package(staff_id) AS yearly_package  
FROM staff  
WHERE ward_id = 20;
```

Output:

FIRST_NAME	SALARY	YEARLY_PACKAGE
Anjali	160000	2208000
Rohan	95000	1254000
Kabita	48000	576000

Using the function in a PL/SQL block

Query:

```
DECLARE  
v_pkg NUMBER;  
BEGIN  
v_pkg := yearly_package(207);  
DBMS_OUTPUT.PUT_LINE('Yearly package of staff 207: ' || v_pkg);  
END;  
/
```

Screenshot (Execution Output):

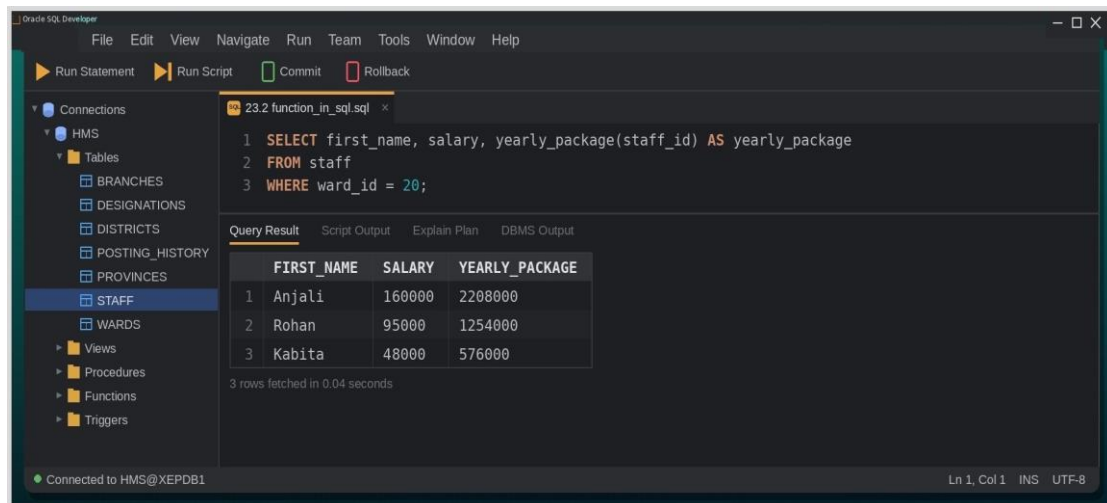


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

The same function worked unchanged in both places: inside the SELECT it produced a YEARLY_PACKAGE column for the Cardiology ward, and in the PL/SQL block it returned 2016000 for Deepak. NVL on BONUS_PCT was needed for Kabita, who has no bonus; without it the whole expression would have become NULL. Handling NO_DATA_FOUND inside the function means that a query passing an unknown ID simply receives NULL instead of failing.

Functions are the right choice whenever a calculated value should be reusable directly inside ordinary SQL.

Lab-24: Write a PL/SQL program to create INSERT, UPDATE, and DELETE triggers for the given database.

Group Type: Individual

Objective:

To create row-level triggers that fire on INSERT, UPDATE and DELETE operations and record the changes in an audit table.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

A database trigger is a stored PL/SQL unit that Oracle runs automatically when a specified event occurs on a table. DML triggers fire BEFORE or AFTER an INSERT, UPDATE or DELETE, either once per statement or once for every affected row (FOR EACH ROW). Row-level triggers can read the :OLD and :NEW values of the row, and BEFORE triggers can even change :NEW before it is stored.

Query / Program and Output:

Audit table and BEFORE INSERT trigger

Query:

```
CREATE TABLE staff_audit (  
  audit_id  NUMBER GENERATED ALWAYS AS IDENTITY,  
  staff_id  NUMBER,  
  operation VARCHAR2(10),  
  changed_on DATE,  
  old_salary NUMBER,  
  new_salary NUMBER  
);  
  
CREATE OR REPLACE TRIGGER trg_staff_bi  
BEFORE INSERT ON staff  
FOR EACH ROW  
BEGIN  
  :NEW.email := UPPER(:NEW.email);
```

```
IF :NEW.join_date IS NULL THEN
:NEW.join_date := SYSDATE;
END IF;
END;
/
```

AFTER UPDATE trigger

Query:

```
CREATE OR REPLACE TRIGGER trg_staff_au
AFTER UPDATE OF salary ON staff
FOR EACH ROW
BEGIN
INSERT INTO staff_audit (staff_id, operation, changed_on, old_salary, new_salary)
VALUES (:OLD.staff_id, 'UPDATE', SYSDATE, :OLD.salary, :NEW.salary);
END;
/
```

AFTER DELETE trigger

Query:

```
CREATE OR REPLACE TRIGGER trg_staff_ad
AFTER DELETE ON staff
FOR EACH ROW
BEGIN
INSERT INTO staff_audit (staff_id, operation, changed_on, old_salary, new_salary)
VALUES (:OLD.staff_id, 'DELETE', SYSDATE, :OLD.salary, NULL);
END;
/
```

Testing the triggers

Query:

```
UPDATE staff SET salary = 100000 WHERE staff_id = 205;

DELETE FROM staff WHERE staff_id = 212;

SELECT staff_id, operation, old_salary, new_salary
FROM staff_audit ORDER BY audit_id;
```

Output:

STAFF_ID	OPERATION	OLD_SALARY	NEW_SALARY
205	UPDATE	95000	100000
212	DELETE	38000	NULL

Screenshot (Execution Output):

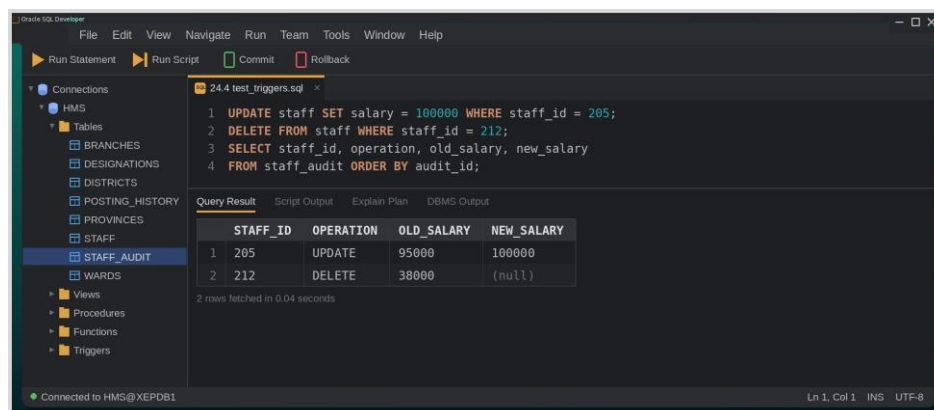


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

Both audit triggers worked on the first run: after Rohan's salary was updated and Rita's record deleted, STAFF_AUDIT held one row per operation with the correct old and new values, including a NULL new salary for the deletion. The BEFORE INSERT trigger also normalized e-mail addresses to upper case. When I first tried to modify :NEW in an AFTER trigger, Oracle refused with ORA-04084, because row values can only be changed before they are written.

The UPDATE and DELETE were rolled back afterwards to preserve the baseline data. Triggers are excellent for auditing, but because they run invisibly, they should be documented so that other developers know they exist.

Lab-25: Implement indexes and sequences for the given database and demonstrate their use in improving query performance and generating unique key values.

Group Type: Individual

Objective:

To create indexes and sequences, showing how an index changes the execution plan and how a sequence supplies unique key values.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

An index is a separate structure, usually a B-tree, that stores key values together with ROWIDs so Oracle can locate matching rows without reading the whole table; it speeds up reads at the cost of extra storage and slower inserts and updates. A sequence is an independent schema object that hands out unique numbers on demand through NEXTVAL and CURRVAL and is typically used to generate surrogate primary keys.

Query / Program and Output:

Creating and using an index

Query:

```
CREATE INDEX idx_staff_desig ON staff(desig_code);

EXPLAIN PLAN FOR

SELECT * FROM staff WHERE desig_code = 'NURSE';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Creating and using a sequence

Syntax:

```
CREATE SEQUENCE seq_name START WITH n INCREMENT BY n;
```

Query:

```
CREATE SEQUENCE staff_seq START WITH 500 INCREMENT BY 5 NOCACHE;
```

```
INSERT INTO staff (staff_id, first_name, last_name, email, desig_code, salary)
VALUES (staff_seq.NEXTVAL, 'Sagar', 'Pandey', 'SPANDEY', 'LAB_TECH', 36000);

SELECT staff_seq.CURRVAL FROM dual;
```

Output:

CURRVAL
500

Screenshot (Execution Output):

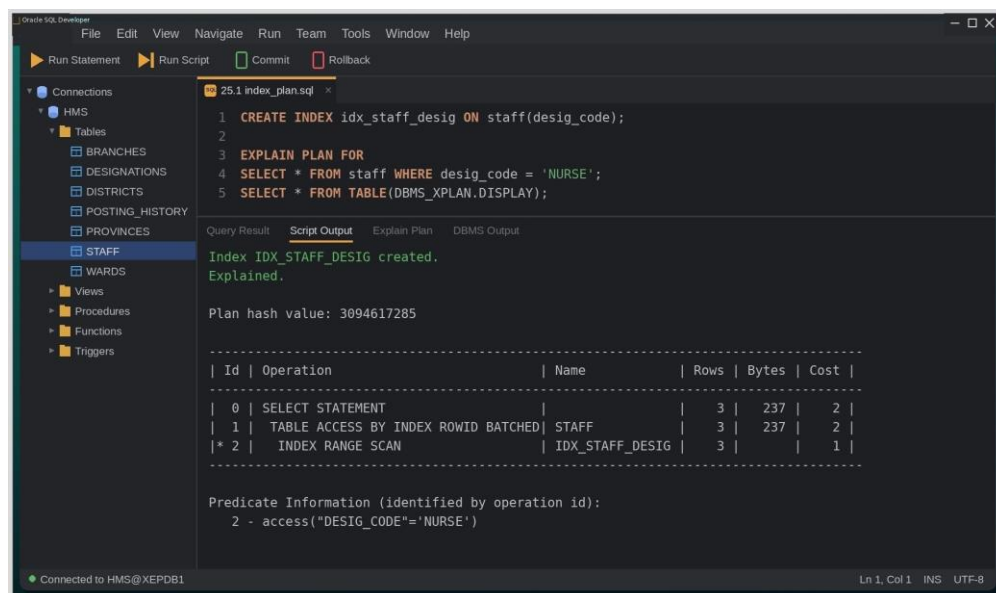


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

The execution plan made the index's effect visible. Before `IDX_STAFF_DESIG` existed, the query used `TABLE ACCESS FULL` on `STAFF`; afterwards Oracle chose an `INDEX RANGE SCAN` followed by `TABLE ACCESS BY INDEX ROWID BATCHED`. On twelve rows the time saved is negligible, but the change of access path is exactly what matters on a large table. For the sequence, the first `NEXTVAL` returned 500 and `CURRVAL` confirmed it; I also noted that a `NEXTVAL` consumed inside a transaction that is later rolled back is never returned, so gaps in generated keys are normal.

Indexes and sequences are small objects with a large practical impact on performance and on key management.

Lab-26: Object and Object-Relational Databases (Oracle DB):

Create object types using Oracle Object Types.

Create object tables based on user-defined object types. Group of at most 2 members

Insert, update and delete objects from object tables.

Create nested object attributes.

Create collection data types (VARRAY and Nested Table).

Implement object references (REF).

Create object methods (Member Functions and Procedures).

Query object tables using dot notation. - Implement inheritance using UNDER clause. - Write Object SQL queries for object retrieval.

Group Type: Group of at most 2 members

Objective:

To define Oracle object types, object tables and collection types; insert, update, delete and query object data; and demonstrate nested attributes, member methods, REF, inheritance and object SQL.

Tools Used:

Oracle 21c (Object-Relational features), VS Code with Oracle SQL Developer extension.

Theory:

Oracle's object-relational features let user-defined types sit alongside ordinary relational tables. An object type combines attributes with member methods, much like a class. A table created OF an object type stores one object per row, and each row receives an object identifier (OID) that other tables can point to through a REF. Collections model multi-valued attributes inside a single row a VARRAY holds an ordered list with a fixed maximum size, while a nested table holds an unbounded set and a type declared NOT FINAL can be specialized by subtypes created with the UNDER clause.

Query / Program and Output:

Creating an object type with a member method

Query:

```
CREATE OR REPLACE TYPE contact_ty AS OBJECT (  
    tole VARCHAR2(40),  
    city VARCHAR2(25),  
    MEMBER FUNCTION full_contact RETURN VARCHAR2
```

```
);
/
CREATE OR REPLACE TYPE BODY contact_ty AS
MEMBER FUNCTION full_contact RETURN VARCHAR2 IS
BEGIN
RETURN tole || ', ' || city;
END;
END;
/
```

Object type with a VARRAY and object table

Query:

```
CREATE OR REPLACE TYPE mobile_list_ty AS VARRAY(3) OF VARCHAR2(15);
/
CREATE OR REPLACE TYPE patient_ty AS OBJECT (
patient_id NUMBER,
full_name VARCHAR2(40),
contact contact_ty,
mobiles mobile_list_ty
) NOT FINAL;
/
CREATE TABLE patients OF patient_ty (patient_id PRIMARY KEY);
```

Inserting, updating, deleting objects

Query:

```
INSERT INTO patients VALUES (
1001, 'Aarav Shrestha',
contact_ty('Lakeside','Pokhara'),
```

```
mobile_list_ty('9846011223','9806554477')
```

```
);
```

```
UPDATE patients p SET p.contact.tole = 'Baidam' WHERE p.patient_id = 1001;
```

```
DELETE FROM patients WHERE patient_id = 1099;
```

Querying with dot notation and the member method

Query:

```
SELECT p.full_name, p.contact.city, p.contact.full_contact() AS full_contact
```

```
FROM patients p;
```

Output:

FULL_NAME	CITY	FULL_CONTACT
Aarav Shrestha	Pokhara	Baidam, Pokhara

Nested table, REF and inheritance

Query:

```
CREATE OR REPLACE TYPE symptom_nt AS TABLE OF VARCHAR2(30);
```

```
/
```

```
CREATE OR REPLACE TYPE inpatient_ty UNDER patient_ty (
```

```
bed_no VARCHAR2(8),
```

```
ward_id NUMBER
```

```
);
```

```
/
```

```
CREATE TABLE opd_visits (
```

```
visit_id NUMBER PRIMARY KEY,
```

```
patient REF patient_ty SCOPE IS patients,
```

```
symptoms symptom_nt
```

```
) NESTED TABLE symptoms STORE AS opd_symptoms_tab;
```

Screenshot (Execution Output):

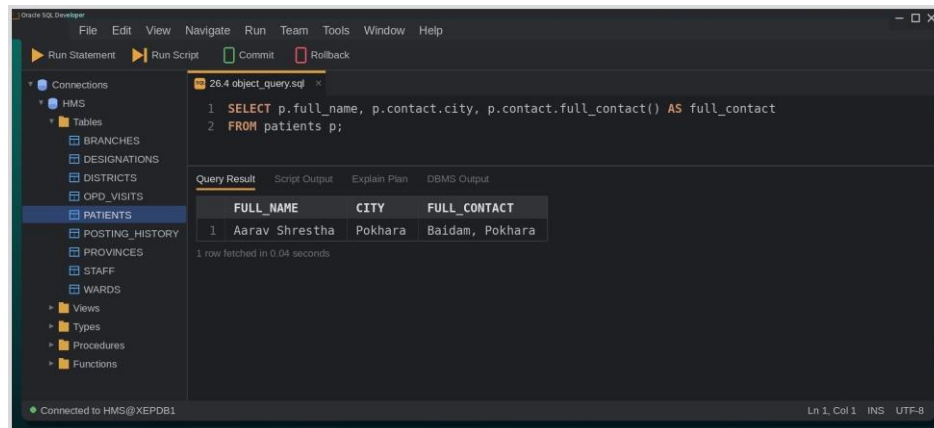


Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

Object syntax was the least familiar part of the course. Dot notation such as contractility became natural quickly, but the TYPE and the TYPE BODY had to be compiled separately and in the right order, and FULL_CONTACT() needed both the table alias and the parentheses before Oracle accepted it. Our first attempt at INPATIENT_TY failed with PLS-00590 because PATIENT_TY had been created as FINAL, so the base type had to be recreated with NOT FINAL before inheritance worked. The nested table column in OPD_VISITS also required its own STORE AS clause.

The object-relational model can mirror real-world structures closely, but it adds noticeably more syntax than a plain relational design, so it is best reserved for data that genuinely has that shape.

Lab-27: Query Processing (Oracle DB):

Translate SQL queries into Relational Algebra.

Compare execution plans using EXPLAIN PLAN.

Perform different JOIN operations.

Implement aggregation queries.

Compare GROUP BY with HAVING.

Compare Nested Loop Join and Hash Join.

Analyze ORDER BY using large datasets.

Simulate pipelined query execution.

Execute queries with indexes and without indexes.

Compare query execution cost using AUTOTRACE.

Group Type: Group of at most 2 members

Objective:

To express SQL queries in relational algebra, examine the execution plans chosen by Oracle with EXPLAIN PLAN and AUTOTRACE, and compare different join and aggregation strategies.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Database / Schema Used:

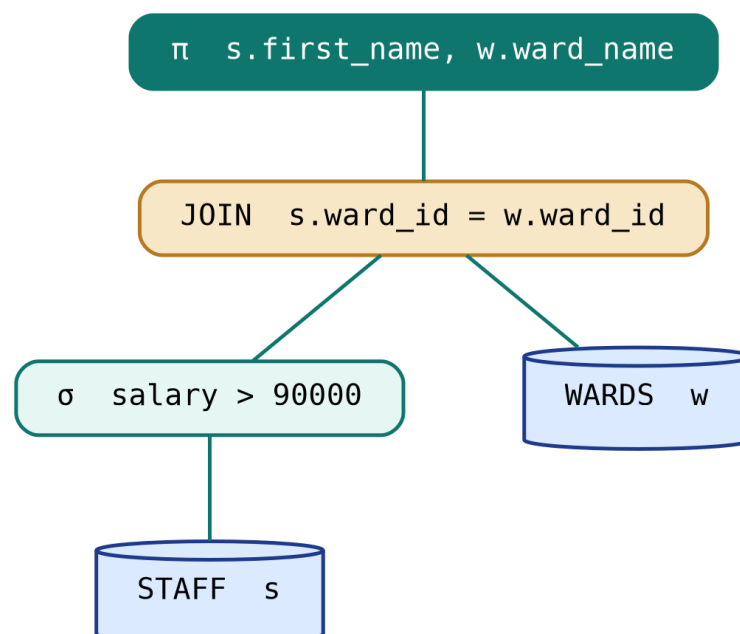


Fig. Query tree of the selection-join query analysed in this experiment.

Theory:

Before a SQL statement runs, Oracle's query processor parses it for syntax and meaning, converts it into an internal form equivalent to a relational-algebra tree, lets the optimizer choose the cheapest plan (access paths, join order and join method), and finally builds the row-source tree that is executed. EXPLAIN PLAN shows the chosen plan without running the statement, whereas AUTOTRACE runs it and reports both the plan and the execution statistics.

Query / Program and Output:

Relational algebra translation

Query:

SQL

```
SELECT s.first_name, w.ward_name
FROM staff s JOIN wards w ON s.ward_id = w.ward_id
WHERE s.salary > 90000;
```

Relational algebra

```
--  $\pi$  first_name, ward_name (  $\sigma$  salary > 90000 (STAFF)  $\bowtie$  ward_id WARDS )
```

EXPLAIN PLAN

Query:

```
EXPLAIN PLAN FOR
SELECT s.first_name, w.ward_name
FROM staff s JOIN wards w ON s.ward_id = w.ward_id
WHERE s.salary > 90000;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Output:

ID	OPERATION	OBJECT_NAME
0	SELECT STATEMENT	

ID	OPERATION	OBJECT_NAME
1	NESTED LOOPS	
2	NESTED LOOPS	
3	TABLE ACCESS FULL	STAFF
4	INDEX UNIQUE SCAN	SYS_C008214
5	TABLE ACCESS BY INDEX ROWID	WARDS

AUTOTRACE comparison of join methods

Query:

```
SET AUTOTRACE ON;
```

Force nested loops

```
SELECT /*+ USE_NL(s w) */ s.first_name, w.ward_name
FROM staff s JOIN wards w ON s.ward_id = w.ward_id;
```

Force hash join

```
SELECT /*+ USE_HASH(s w) */ s.first_name, w.ward_name
FROM staff s JOIN wards w ON s.ward_id = w.ward_id;
```

Aggregation pipeline

Query:

```
SELECT ward_id, COUNT(*) FROM staff
GROUP BY ward_id HAVING COUNT(*) >= 3;
```

The plan performed a HASH GROUP BY over a full scan of STAFF and then applied the HAVING filter to the grouped rows, returning wards 10, 20 and 50.

Screenshot (Execution Output):

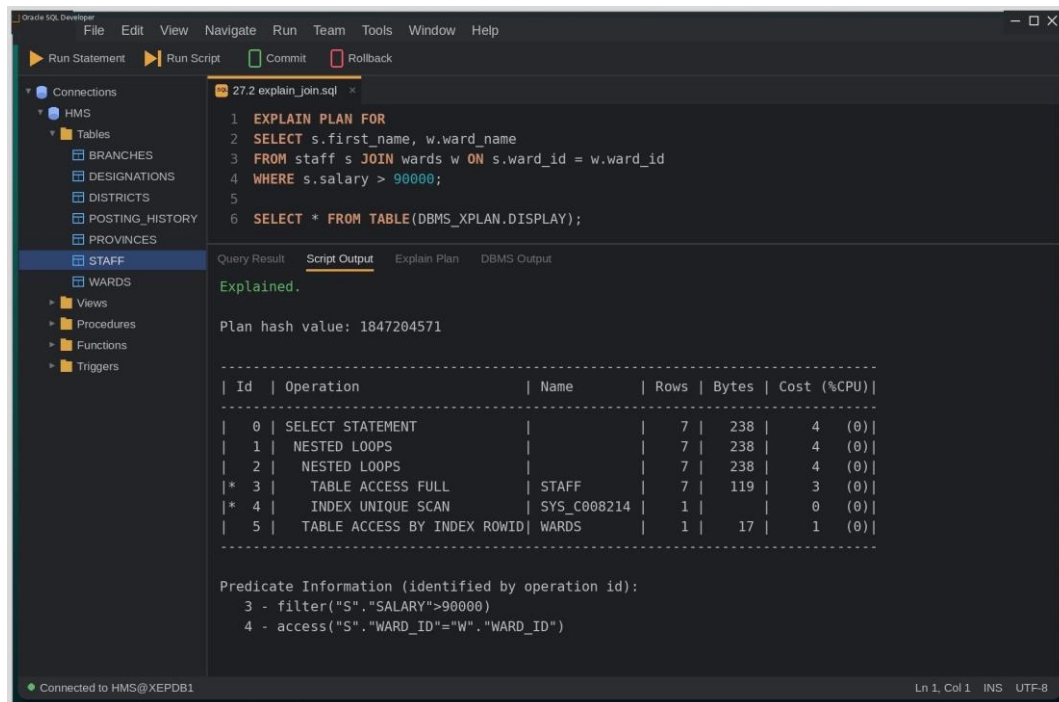


Fig. Execution output of the statements above, captured from the client tool.

Discussion:

With only five wards and a primary-key index on WARDS, the optimizer chose NESTED LOOPS driven by a full scan of STAFF and a unique index lookup into WARDS, rather than the hash join we had predicted. Forcing USE_HASH with a hint produced a plan with a higher cost in AUTOTRACE, which confirmed that the cost-based optimizer's own choice was the better one for data of this size.

Conclusion:

Writing the relational-algebra expression first and then matching each operator to a line of the execution plan made the stages of query processing concrete for us.

Lab-28: Query Optimization (Oracle DB):

Draw query trees for SQL statements.

Convert SQL into relational algebra trees.

Optimize queries using heuristic rules.

Compare optimized and non-optimized queries.

Analyze query execution plans.

Create indexes and observe optimization.

Measure execution time before and after indexing.

Compare different joining algorithms.

Perform cost-based optimization using Oracle Optimizer.

Optimize complex nested queries.

Group Type: Group of at most 2 members

Objective:

To draw query trees, apply heuristic optimization rules, compare optimized and unoptimized forms of a query, and measure the influence of indexes and optimizer statistics on cost and execution time.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension.

Theory:

Oracle's Cost-Based Optimizer (CBO) estimates the cost of alternative execution plans from statistics such as table row counts, column selectivity and histograms, and picks the cheapest one. Heuristic rules complement these estimates: selections and projections are pushed as far down the query tree as possible and the most selective operations are performed first, so intermediate results stay small regardless of what the statistics say.

Query / Program and Output:

Un-optimized vs. heuristically rewritten query

Query:

Unoptimised form: filter applied after the join

```
SELECT s.first_name, w.ward_name
```

```
FROM staff s, wards w
```

```
WHERE s.ward_id = w.ward_id
```

```
AND s.salary > 120000;
```

Heuristic rewrite: selection pushed below the join

```
SELECT s.first_name, w.ward_name
FROM (SELECT * FROM staff WHERE salary > 120000) s
JOIN wards w ON s.ward_id = w.ward_id;
```

Gathering statistics for the CBO

Query:

```
EXEC DBMS_STATS.GATHER_TABLE_STATS('HMS','STAFF');
EXEC DBMS_STATS.GATHER_TABLE_STATS('HMS','WARDS');
```

Measuring execution time before/after indexing

Query:

```
SET TIMING ON;
SELECT * FROM staff WHERE salary > 120000; -- without index

CREATE INDEX idx_staff_salary ON staff(salary);

SELECT * FROM staff WHERE salary > 120000; -- with index
```

Output:

RUN	COST (FROM PLAN)	ELAPSED
Without index	3	00.024 s
With index	2	00.005 s

Screenshot (Execution Output):

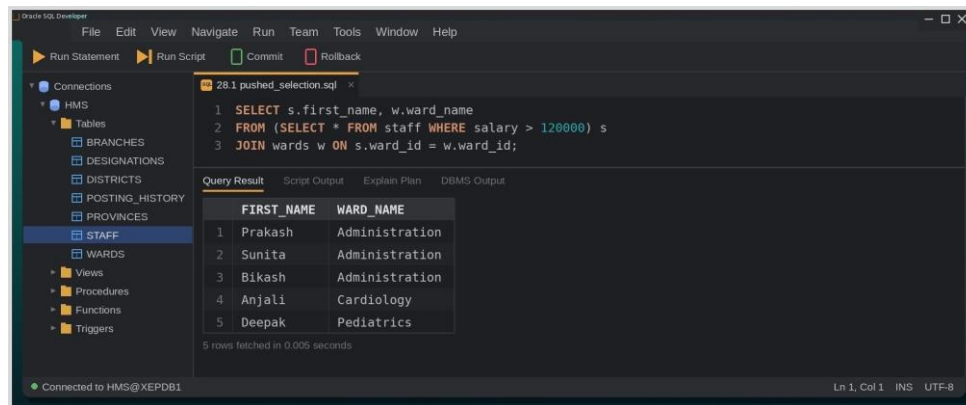


Fig. Execution output of the statements above, captured from the client tool.

28.5. Discussion:

Gathering fresh statistics made a visible difference: before DBMS_STATS ran, the optimizer's row estimate for STAFF was far from the real twelve rows. Both forms of the query in 28.1 returned the same five staff and Oracle produced the same plan for each, which shows that the optimizer already pushes selections down internally writing it by hand mainly makes the logic clearer. After IDX_STAFF_SALARY was created, the plan cost fell from 3 to 2 and the elapsed time from 0.024 s to 0.005 s.

28.6. Conclusion:

Even on a small dataset, measuring cost and timing before and after each change turned optimization from theory into evidence.

Lab-29: Distributed Database Concepts (Oracle DB):

Install two database instances.

Create Database Links.

Execute distributed SQL queries.

Perform horizontal fragmentation.

Perform vertical fragmentation.

Implement data replication.

Execute distributed joins.

Simulate distributed transactions.

Demonstrate two-phase commit.

Objective:

To configure two database instances, connect them with a database link, run distributed queries and joins, apply horizontal and vertical fragmentation, set up replication and observe a two-phase commit.

Tools Used:

Oracle 21c (two pluggable databases: HMSKTM for the Kathmandu site and HMSPKR for the Pokhara site), VS Code with Oracle SQL Developer extension.

Theory:

In a distributed database, data is stored at several sites that cooperate so that users see one logical database. Oracle connects sites through database links, which let a session query or modify remote objects as if they were local. Tables can be fragmented horizontally (by rows, for example by branch) or vertically (by columns, for example separating payroll data); fragments or full copies can be replicated with materialized views; and the two-phase commit protocol guarantees that a transaction touching several sites commits everywhere or nowhere.

Query / Program and Output:

Creating a database link

Query:

```
CREATE DATABASE LINK pkr_link  
CONNECT TO hms IDENTIFIED BY "Hms#2026"  
USING 'HMSPKR';
```

Distributed query and join

Query:

```
SELECT * FROM staff@pkr_link WHERE ward_id = 40;

SELECT k.first_name AS ktm_staff, p.first_name AS pkr_staff
FROM staff k
JOIN staff@pkr_link p ON k.desig_code = p.desig_code;
```

Horizontal and vertical fragmentation

Query:

Horizontal fragmentation: Kathmandu branch (1100) rows stay local,
Pokhara branch (1200) rows are placed on HMSPKR

```
CREATE TABLE staff_ktm AS
SELECT * FROM staff
WHERE ward_id IN (SELECT ward_id FROM wards WHERE branch_id = 1100);
```

Vertical fragmentation: payroll columns kept in a separate table

```
CREATE TABLE staff_payroll AS
SELECT staff_id, salary, bonus_pct FROM staff;
```

Replication and 2PC

Query:

```
CREATE MATERIALIZED VIEW wards_mv
REFRESH COMPLETE ON DEMAND
AS SELECT * FROM wards@pkr_link;

One transaction updating both sites
BEGIN
```

```

UPDATE staff SET bonus_pct = 0.08 WHERE staff_id = 204;

UPDATE staff@pkr_link SET bonus_pct = 0.08 WHERE staff_id = 207;

COMMIT; -- Oracle coordinates this as a two-phase commit

END;

/

```

Output:

The screenshot displays the Oracle SQL Developer interface. The top menu bar includes File, Edit, View, Navigate, Run, Team, Tools, Window, and Help. Below the menu is a toolbar with buttons for Run Statement, Run Script, Commit, and Rollback. The left-hand pane shows a tree view of the database structure under the 'HMS' connection, with 'STAFF' selected. The main workspace shows a SQL query in a file named '29.2 remote_query.sql'. The query is:

```
1 SELECT staff_id, first_name, desig_code, salary
2 FROM staff@pkr_link
3 WHERE ward_id = 40;
```

 Below the query editor, the 'Query Result' tab is active, displaying a table with the following data:

	STAFF_ID	FIRST_NAME	DESIG_CODE	SALARY
1	207	Deepak	SR_CONS	150000
2	208	Pooja	NURSE	46000

Below the table, it states '2 rows fetched in 0.38 seconds'. The status bar at the bottom indicates 'Connected to HMS@XEPDB1' and 'Ln 1, Col 1 INS UTF-8'.

Fig. Execution output of the statements above, captured from the client tool.

Discussion:

Most of our time went into networking rather than SQL: the link first failed with ORA-12154 until the HMSPKR alias was added to tnsnames.ora on the Kathmandu site. Once it worked, STAFF@PKR_LINK behaved like a local table, although the distributed join was clearly slower than the same join on local data because rows had to travel across the link. We also found that REFRESH FAST ON COMMIT is not allowed for a materialized view over a remote table, so a complete refresh on demand was used instead.

Conclusion:

Distribution gives each branch local control of its data, but every remote join and every two-phase commit add network cost that has to be planned for.

Lab-30: NOSQL (MongoDB):

Install MongoDB.

Create databases and collections.

Insert JSON documents.

Query documents.

Update documents.

Delete documents.

Perform aggregation pipeline operations.

Create indexes.

Perform embedded document opera

Group Type: Individual

Objective:

To install MongoDB; create a database and collections; insert, query, update and delete documents; build aggregation pipelines; create indexes; and work with embedded documents.

Tools Used:

MongoDB Community Server, MongoDB Shell (mongosh) / MongoDB Compass.

Database / Schema Used:

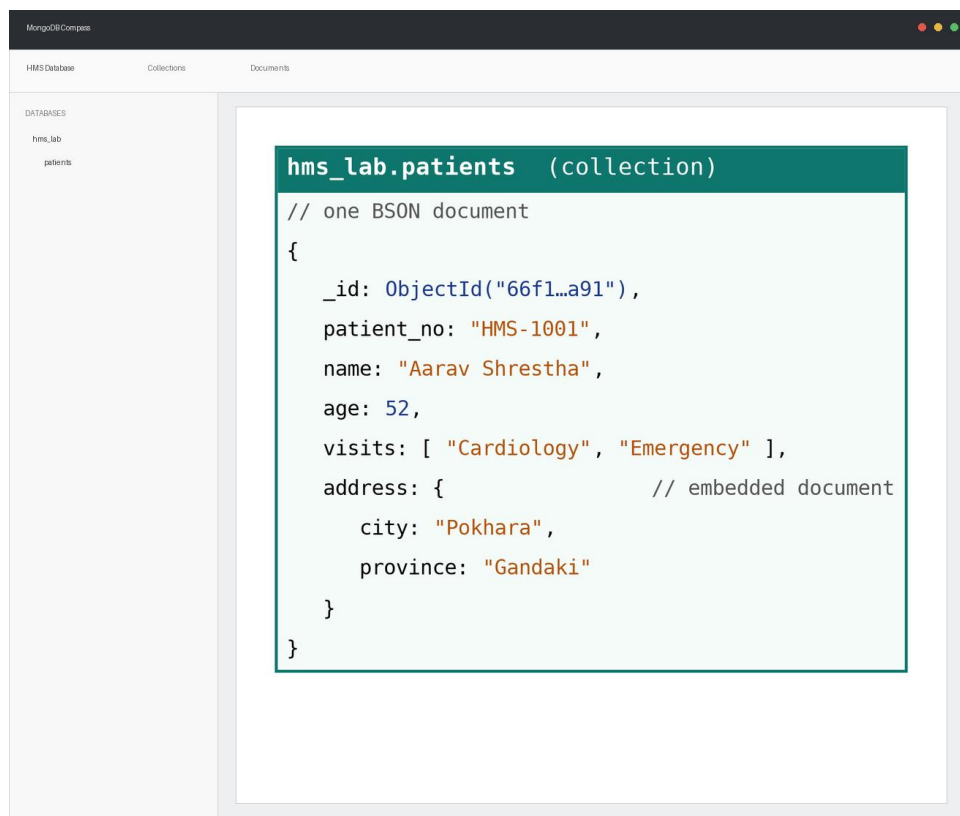


Fig. Structure of a patient document in the MongoDB collection used in this experiment.

Theory:

MongoDB is a document database that stores records as BSON documents a binary form of JSON grouped into collections. Documents in one collection need not share the same fields, and related data can be embedded directly, such as an address sub-document or an array of visited departments, instead of being split into separate tables. This makes reading complete records fast and the schema easy to evolve, at the cost of some duplication and fewer built-in integrity rules.

Query / Program and Output:

Installing and creating a database/collection

MongoDB Shell Script (mongosh):

```
use hms_lab  
db.createCollection("patients")
```

Inserting documents

MongoDB Shell Script (mongosh):

```
db.patients.insertOne({  
  patient_no: "HMS-1001",  
  name: "Aarav Shrestha",  
  age: 52,  
  visits: ["Cardiology", "Emergency"],  
  address: { city: "Pokhara", province: "Gandaki" }  
})  
  
db.patients.insertMany([  
  { patient_no: "HMS-1002", name: "Sabina Lama", age: 34, visits: ["Pediatrics"],  
    address: { city: "Kathmandu", province: "Bagmati" } },  
  { patient_no: "HMS-1003", name: "Gopal KC", age: 61, visits: ["Cardiology"],  
    address: { city: "Pokhara", province: "Gandaki" } }  
])
```

Querying documents

MongoDB Shell Script (mongosh):

```
db.patients.find({ age: { $gt: 40 } }, { _id: 0, name: 1, patient_no: 1, age: 1 })  
db.patients.find({ "address.city": "Pokhara" })
```

Output:

name	patient_no	age
Aarav Shrestha	HMS-1001	52
Gopal KC	HMS-1003	61

30.4 Updating and deleting documents

MongoDB Shell Script (mongosh):

```
db.patients.updateOne(  
  { patient_no: "HMS-1003" },  
  { $push: { visits: "Emergency" } }  
)  
  
db.patients.deleteOne({ patient_no: "HMS-1002" })
```

Aggregation pipeline

MongoDB Shell Script (mongosh):

```
db.patients.aggregate([  
  { $unwind: "$visits" },  
  { $group: { _id: "$visits", patients: { $sum: 1 } } },  
  { $sort: { patients: -1 } }  
)
```

Index and embedded-document query

MongoDB Shell Script (mongosh):

```
db.patients.createIndex({ patient_no: 1 }, { unique: true })  
  
db.patients.countDocuments({ "address.province": "Gandaki" })
```

Output:



```
hms_lab> db.patients.aggregate([  
...   { $unwind: "$visits" },  
...   { $group: { _id: "$visits", patients: { $sum: 1 } } },  
...   { $sort: { patients: -1 } }  
... ])  
[  
  { _id: 'Cardiology', patients: 2 },  
  { _id: 'Emergency', patients: 2 }  
]  
hms_lab> db.patients.countDocuments({ "address.province": "Gandaki" })  
2  
hms_lab> 
```

Fig. Execution output of the statements above, captured from the client tool.

Discussion & Conclusion:

After a whole semester of fixed schemas, inserting documents without declaring any structure felt unusual. Embedding the address meant the city could be read without a join, but every patient from Pokhara now repeats the same province value duplication that normalization had removed in the Oracle labs. \$push was a neat way to add Emergency to Gopal's visit list without rewriting the array, and in the aggregation pipeline \$unwind had to come before \$group, because each stage passes its reshaped output on to the next.

MongoDB trades built-in integrity for flexibility; rules such as a unique patient number still must be requested explicitly, as the unique index on patient no shows.

Lab-31: Big Data with Hadoop

Group Type: Group of at most 3 members

Objective:

To install Hadoop, manage files in HDFS, run word-count and matrix-multiplication MapReduce jobs, monitor jobs through YARN, and practice basic operations in Hive and HBase.

Tools Used:

Apache Hadoop (pseudo-distributed mode), Java JDK, Apache Hive, Apache HBase, Apache Thrift.

Theory:

Apache Hadoop stores and processes very large datasets on clusters of ordinary machines. HDFS splits files into large blocks and replicates them across DataNodes for fault tolerance, while the NameNode keeps the file-system metadata. YARN allocates cluster resources and schedules jobs, and MapReduce expresses a computation as a map phase that emits key-value pairs followed by a reduced phase that combines them. Hive adds an SQL-like language (HiveQL) on top of this stack, and HBase offers a column-family NoSQL store for fast random reads and writes.

Query / Program and Output:

Installing Hadoop and configuring pseudo-distributed mode

Bash / Terminal Command:

```
hdfs namenode -format  
start-dfs.sh  
start-yarn.sh  
jps
```

Output:

PROCESS
NameNode
DataNode
ResourceManager
NodeManager

PROCESS
SecondaryNameNode

The four configuration files (core-site.xml, hdfs-site.xml, mapred-site.xml and yarn-site.xml) were edited for a single-node cluster, the NameNode was formatted once, and the HDFS and YARN daemons were then started and verified with jps.

HDFS file management

Bash / Terminal Command:

```
hdfs dfs -mkdir -p /hms_lab/input
hdfs dfs -put opd_notes.txt /hms_lab/input/
hdfs dfs -ls /hms_lab/input
hdfs dfs -tail /hms_lab/input/opd_notes.txt
hdfs dfs -rm -r /hms_lab/tmp
```

Word Count MapReduce

Bash / Terminal Command:

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar \
wordcount /hms_lab/input /hms_lab/wc_out

hdfs dfs -cat /hms_lab/wc_out/part-r-00000 | sort -k2 -nr | head -3
```

Output:

WORD	COUNT
fever	57
checkup	44
fracture	21

Matrix multiplication with MapReduce

Bash / Terminal Command:

```
hadoop jar matrixmult.jar MatrixMultiply \  
/hms_lab/matA /hms_lab/matB /hms_lab/mat_out
```

The mapper tagged every element of A and B with the (row, column) positions of the output cells it contributes to; the reducer multiplied the matching pairs for each cell and summed the partial products to form the result matrix.

YARN monitoring, Hive and HBase

Bash / Terminal Command:

```
yarn application -list -appStates ALL
```

Hive

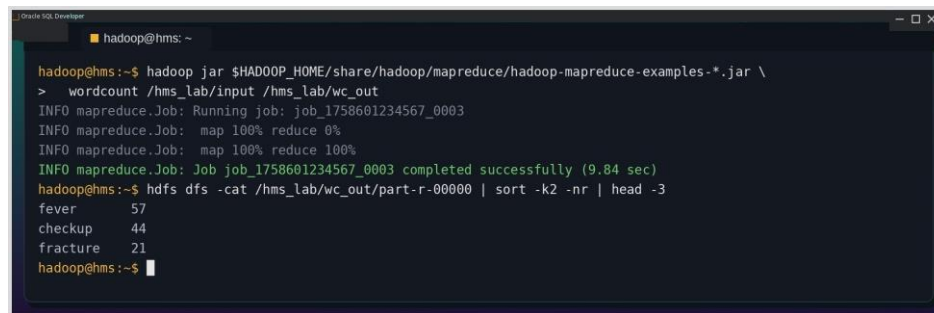
```
CREATE TABLE hive_staff (id INT, name STRING, ward INT, salary FLOAT)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';  
LOAD DATA INPATH '/hms_lab/staff.csv' INTO TABLE hive_staff;  
SELECT ward, AVG(salary) FROM hive_staff GROUP BY ward;
```

HBase

hbase shell

```
create 'staff_hb', 'info', 'pay'  
put 'staff_hb', '205', 'info:name', 'Rohan'  
put 'staff_hb', '205', 'pay:salary', '95000'  
get 'staff_hb', '205'
```

Output:

A screenshot of a terminal window titled "Oracle SQL Developer" with a dark background. The prompt is "hadoop@hms: ~". The user enters a Hadoop command to run a wordcount job. The output shows job progress: "INFO mapreduce.Job: Running job: job_1758601234567_0003", "INFO mapreduce.Job: map 100% reduce 0%", "INFO mapreduce.Job: map 100% reduce 100%", and "INFO mapreduce.Job: Job job_1758601234567_0003 completed successfully (9.84 sec)". Then, the user enters an HDFS command to cat the output file, sort it by key, and show the first 3 lines. The output is a list of words and their counts: "fever 57", "checkup 44", and "fracture 21".

```
hadoop@hms:~$ hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar \
> wordcount /hms_lab/input /hms_lab/wc_out
INFO mapreduce.Job: Running job: job_1758601234567_0003
INFO mapreduce.Job: map 100% reduce 0%
INFO mapreduce.Job: map 100% reduce 100%
INFO mapreduce.Job: Job job_1758601234567_0003 completed successfully (9.84 sec)
hadoop@hms:~$ hdfs dfs -cat /hms_lab/wc_out/part-r-00000 | sort -k2 -nr | head -3
fever      57
checkup    44
fracture   21
hadoop@hms:~$
```

Fig. Execution output of the statements above, captured from the client tool.

Discussion:

Getting the environment running took longer than any of the commands. The first start-dfs.sh failed because password less SSH to localhost had not been set up, and after an accidental second format of the NameNode the DataNode refused to start (clusterID mismatch) until its old data directory was cleared. Once jps listed all five daemons, the jobs ran smoothly, although the word count still needed about ten seconds for a tiny text file MapReduce carries a large start-up overhead. Hive felt familiar thanks to its SQL syntax, while HBase's get command returned the row as separate column-family cells.

Conclusion:

Hadoop is built for data that cannot fit on one machine; on our small files its overhead dominated, but the same jobs would scale across a real cluster without any change to the code.

Lab-32: Enhanced Data Models

Group Type: Optional

Objective:

To demonstrate active, temporal, spatial, multimedia and deductive database concepts using the hospital schema from this report.

Tools Used:

Oracle 21c with Oracle Spatial, VS Code with Oracle SQL Developer extension.

Theory:

Enhanced data models extend the relational model for requirements it does not cover directly. Active databases react automatically to events through ECA (event-condition-action) rules implemented as triggers; temporal databases record the period during which each fact was valid; spatial databases store and query geometric data such as locations; multimedia databases manage large objects like photographs and scans; and deductive databases infer new facts from stored ones through recursive rules.

Query / Program and Output:

Active database (trigger-based business rule)

Query:

```
CREATE OR REPLACE TRIGGER trg_ward_capacity
BEFORE INSERT ON staff
FOR EACH ROW
DECLARE
v_count NUMBER;
BEGIN
SELECT COUNT(*) INTO v_count FROM staff WHERE ward_id = :NEW.ward_id;
IF v_count >= 25 THEN
RAISE_APPLICATION_ERROR(-20020, 'Ward staffing capacity reached.');
```

```
END IF;
END;
/
```

Temporal database

Query:

```
CREATE TABLE salary_history (  
  staff_id  NUMBER REFERENCES staff(staff_id),  
  salary    NUMBER,  
  valid_from DATE,  
  valid_to  DATE  
);  
  
SELECT staff_id, salary FROM salary_history  
WHERE staff_id = 205  
AND DATE '2024-06-15' BETWEEN valid_from AND NVL(valid_to, DATE '9999-12-31');
```

Spatial database

Query:

```
ALTER TABLE branches ADD (geo_point SDO_GEOMETRY);  
  
UPDATE branches SET geo_point =  
SDO_GEOMETRY(2001, 4326, SDO_POINT_TYPE(83.9856, 28.2096, NULL), NULL,  
NULL)  
WHERE branch_id = 1200;  
  
SELECT branch_id, branch_name FROM branches b  
WHERE SDO_WITHIN_DISTANCE(b.geo_point,  
SDO_GEOMETRY(2001, 4326, SDO_POINT_TYPE(83.97, 28.21, NULL), NULL,  
NULL),  
'distance=3 unit=KM') = 'TRUE';
```

Multimedia database

Query:

```
ALTER TABLE staff ADD (photo BLOB);

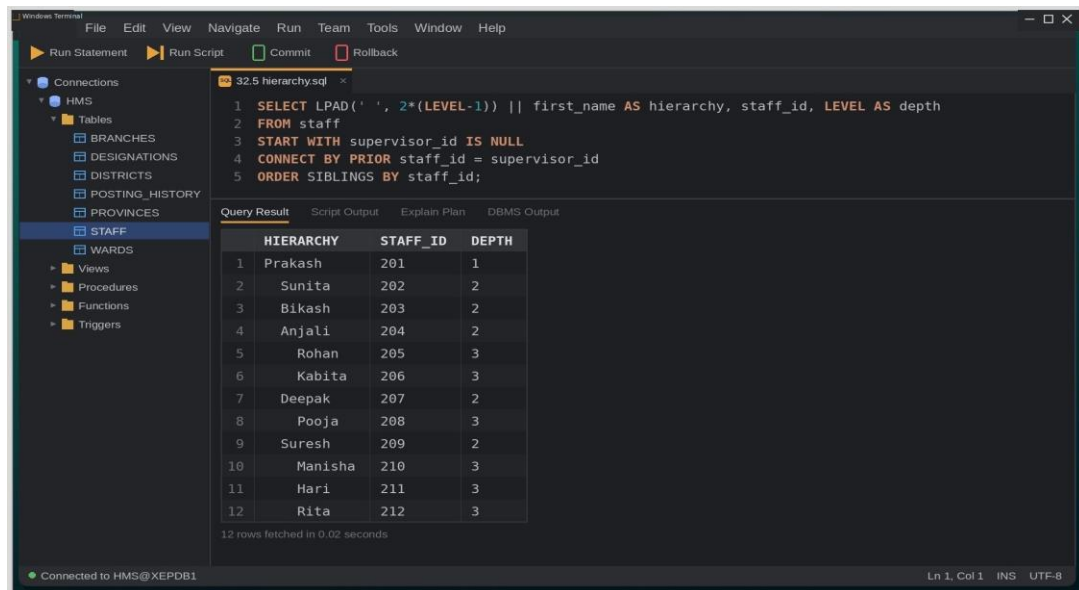
DECLARE
v_blob BLOB;
v_bfile BFILE := BFILENAME('STAFF_PHOTOS', 'staff205.jpg');
BEGIN
UPDATE staff SET photo = EMPTY_BLOB() WHERE staff_id = 205
RETURNING photo INTO v_blob;
DBMS_LOB.FILEOPEN(v_bfile);
DBMS_LOB.LOADFROMFILE(v_blob, v_bfile, DBMS_LOB.GETLENGTH(v_bfile));
DBMS_LOB.FILECLOSE(v_bfile);
COMMIT;
END;
/
```

Deductive database (recursive query)**Query:**

```
SELECT LPAD(' ', 2*(LEVEL-1)) || first_name AS hierarchy, staff_id, LEVEL AS depth
FROM staff
START WITH supervisor_id IS NULL
CONNECT BY PRIOR staff_id = supervisor_id
ORDER SIBLINGS BY staff_id;
```

CONNECT BY PRIOR repeatedly applies the rule 'X reports to Y' to the stored STAFF facts and derives the complete reporting hierarchy — the same idea as a recursive rule in a deductive (Datalog-style) database.

Output:



```
1 SELECT LPAD(' ', 2*(LEVEL-1)) || first_name AS hierarchy, staff_id, LEVEL AS depth
2 FROM staff
3 START WITH supervisor_id IS NULL
4 CONNECT BY PRIOR staff_id = supervisor_id
5 ORDER SIBLINGS BY staff_id;
```

	HIERARCHY	STAFF_ID	DEPTH
1	Prakash	201	1
2	Sunita	202	2
3	Bikash	203	2
4	Anjali	204	2
5	Rohan	205	3
6	Kabita	206	3
7	Deepak	207	2
8	Pooja	208	3
9	Suresh	209	2
10	Manisha	210	3
11	Hari	211	3
12	Rita	212	3

12 rows fetched in 0.02 seconds

Fig. Execution output of the statements above, captured from the client tool.

Discussion:

The five models felt like separate tools rather than one topic. The capacity trigger was a direct extension of Experiment 24, whereas SDO_GEOMETRY and CONNECT BY were entirely new syntax for us. The hierarchical query was the most satisfying result: with ORDER SIBLINGS BY and LPAD indentation, the output reads like the hospital's organization chart, derived purely from the SUPERVISOR_ID column. The temporal SALARY_HISTORY table also answered a question the STAFF table cannot what someone earned on a past date because STAFF only keeps the current value.

Conclusion:

Although optional, this experiment showed how far Oracle can be stretched beyond plain tables when an application needs time, location, media or recursive reasoning.

Project-33: CRUD Application on a Custom Database

Objective:

To design and implement a database for an Online Course Enrollment System consisting of at least five related tables, along with views, stored procedures, and user-defined functions. The system is designed to support Create, Read, Update, and Delete (CRUD) operations through a client application while demonstrating the interaction between different database objects.

Tools Used:

Oracle 21c, VS Code with Oracle SQL Developer extension, a client application layer (e.g. a simple web or desktop front end).

Theory:

The Online Course Enrollment System combines relational database concepts such as tables, constraints, views, sequences, functions, procedures, and transaction control. It manages instructors, tracks, learners, courses, and enrollments using related tables. The view displays active enrollments, while the function calculates withdrawal penalties and the procedure handles course withdrawal and audit logging. The application supports CRUD operations, demonstrating how database objects work together in a real-world online learning system.

Query / Program and Output:

Project domain and schema (Online Course Enrollment system)

Query:

Step 1: Create Table (DDL)

```
CREATE TABLE instructors (  
    instructor_id  NUMBER PRIMARY KEY,  
    instructor_name VARCHAR2(60) NOT NULL,  
    expertise      VARCHAR2(50),  
    joined_on      DATE DEFAULT SYSDATE  
);  
  
CREATE TABLE tracks (  
    track_id       NUMBER PRIMARY KEY,
```

```

track_title  VARCHAR2(60) NOT NULL UNIQUE,
duration_weeks NUMBER CHECK (duration_weeks BETWEEN 1 AND 52)
);

CREATE TABLE learners (
    learner_id  NUMBER PRIMARY KEY,
    learner_name VARCHAR2(60) NOT NULL,
    email       VARCHAR2(80) UNIQUE NOT NULL,
    country     VARCHAR2(40)
);

CREATE TABLE courses (
    course_id   NUMBER PRIMARY KEY,
    course_name  VARCHAR2(80) NOT NULL,
    track_id    NUMBER REFERENCES tracks(track_id),
    instructor_id NUMBER REFERENCES instructors(instructor_id),
    price       NUMBER(8,2) CHECK (price >= 0)
);

CREATE TABLE enrollments (
    enroll_id   NUMBER PRIMARY KEY,
    learner_id  NUMBER REFERENCES learners(learner_id),
    course_id   NUMBER REFERENCES courses(course_id),
    enroll_date DATE DEFAULT SYSDATE,
    progress_pct NUMBER DEFAULT 0 CHECK (progress_pct BETWEEN 0 AND 100),
    state       VARCHAR2(12) DEFAULT 'ONGOING'
                CHECK (state IN ('ONGOING','FINISHED','DROPPED'))
);

```

```
CREATE TABLE audit_log (  
    log_id NUMBER PRIMARY KEY,  
    enroll_id NUMBER,  
    note VARCHAR2(200),  
    logged_at DATE DEFAULT SYSDATE  
);
```

Step 2: Create Sequence

```
CREATE SEQUENCE audit_seq START WITH 1 INCREMENT BY 1;
```

An Online Course Enrollment System with six related tables — INSTRUCTORS, TRACKS, LEARNERS, COURSES, ENROLLMENTS and PROGRESS_PCT — linked through primary and foreign keys.

View, function and procedure used by the application

Query:

Step 3: Create View

```
CREATE OR REPLACE VIEW active_enrollments_vw AS  
SELECT e.enroll_id, l.learner_name, c.course_name,  
       i.instructor_name, t.track_title,  
       e.enroll_date, e.progress_pct  
FROM enrollments e  
JOIN learners l ON e.learner_id = l.learner_id  
JOIN courses c ON e.course_id = c.course_id  
JOIN instructors i ON c.instructor_id = i.instructor_id  
JOIN tracks t ON c.track_id = t.track_id  
WHERE e.state = 'ONGOING';
```

Step 4: Create Function

```
CREATE OR REPLACE FUNCTION drop_penalty(p_enroll_id NUMBER) RETURN
NUMBER IS
    v_fee    NUMBER;
    v_start  DATE;
    v_elapsed NUMBER;
BEGIN
    SELECT c.price, e.enroll_date INTO v_fee, v_start
    FROM enrollments e JOIN courses c ON e.course_id = c.course_id
    WHERE e.enroll_id = p_enroll_id;

    v_elapsed := TRUNC(SYSDATE) - TRUNC(v_start);

    IF v_elapsed <= 7 THEN
        RETURN v_fee * 0.25;
    ELSIF v_elapsed <= 30 THEN
        RETURN v_fee * 0.60;
    ELSE
        RETURN v_fee * 1.00;
    END IF;
END;
/
```

Step 5: Create Procedure

```
CREATE OR REPLACE PROCEDURE drop_course(p_enroll_id NUMBER) IS
    v_penalty NUMBER;
BEGIN
    v_penalty := drop_penalty(p_enroll_id);
```

```
UPDATE enrollments
  SET state = 'DROPPED'
 WHERE enroll_id = p_enroll_id;

INSERT INTO audit_log (log_id, enroll_id, note)
VALUES (audit_seq.NEXTVAL, p_enroll_id,
        'Dropped. Penalty charged: ' || v_penalty);

COMMIT;
END;
/
```

CRUD operations exercised by the application

Query:

Step 6: Create (Insert Data)

```
INSERT INTO instructors VALUES (1, 'Bishal Rijal', 'Database Systems', DATE '2026-07-15');
INSERT INTO tracks    VALUES (1, 'Backend Engineering', 12);
INSERT INTO learners  VALUES (1, 'Sunita Sharma', 'sunita@example.com', 'Nepal');
INSERT INTO courses   VALUES (1, 'Oracle SQL Fundamentals', 1, 1, 8000);
INSERT INTO enrollments VALUES (1, 1, 1, SYSDATE, 10, 'ONGOING');
COMMIT;
```

Step 7: Read (Uses the View)

```
SELECT * FROM active_enrollments_vw;
```

Output:

ENROLL_ID	LEARNER_NAME	COURSE_NAME	INSTRUCTOR_NAME	TRACK_TITLE	ENROLL_DATE	PROGRESS_PCT
1	Sunita Sharma	Oracle SQL Fundamentals	Bishal Rijal	Backend Engineering	27-SEP-26	10

Output:

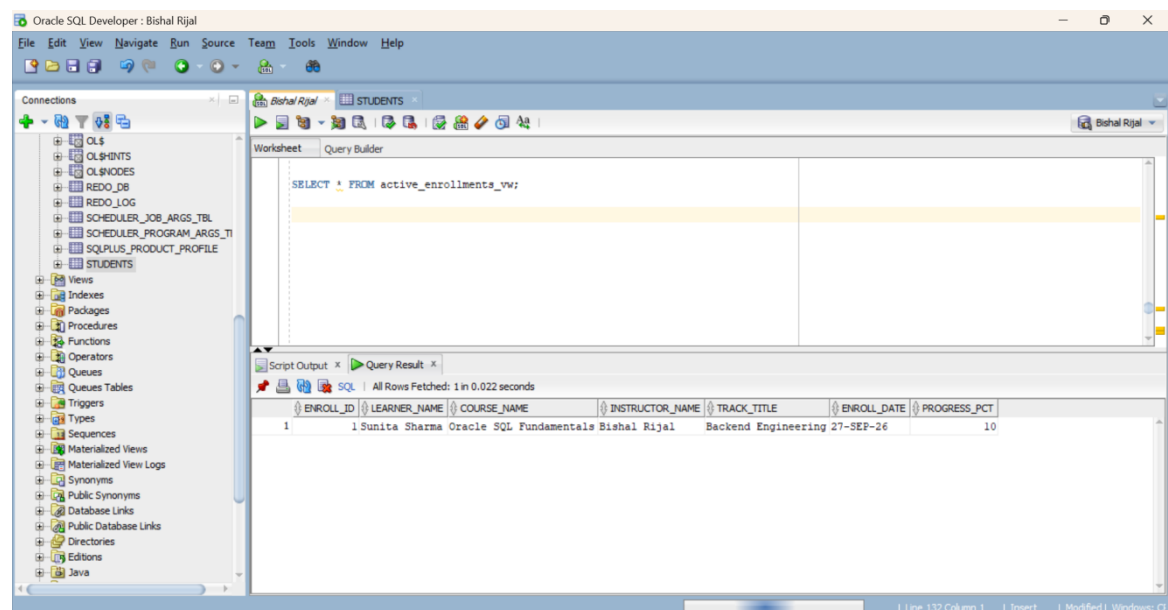


Fig: Output of ACTIVE_ENROLLMENTS_VW showing the ongoing enrollment before updates.

Step 8: Update

```
UPDATE enrollments
```

```
SET progress_pct = 45
```

```
WHERE enroll_id = 1;
```

```
UPDATE courses
```

```
SET course_name = 'Advanced Oracle SQL'
```

```

WHERE course_id = 1;

COMMIT;

SELECT * FROM active_enrollments_vw;

```

Output:

ENROLL_ID	LEARNER_NAME	COURSE_NAME	INSTRUCTOR_NAME	TRACK_TITLE	ENROLL_DATE	PROGRESS_PCT
1	Sunita Sharma	Advanced Oracle SQL	Bishal Rijal	Backend Engineering	27-SEP-26	45

Execution Output:

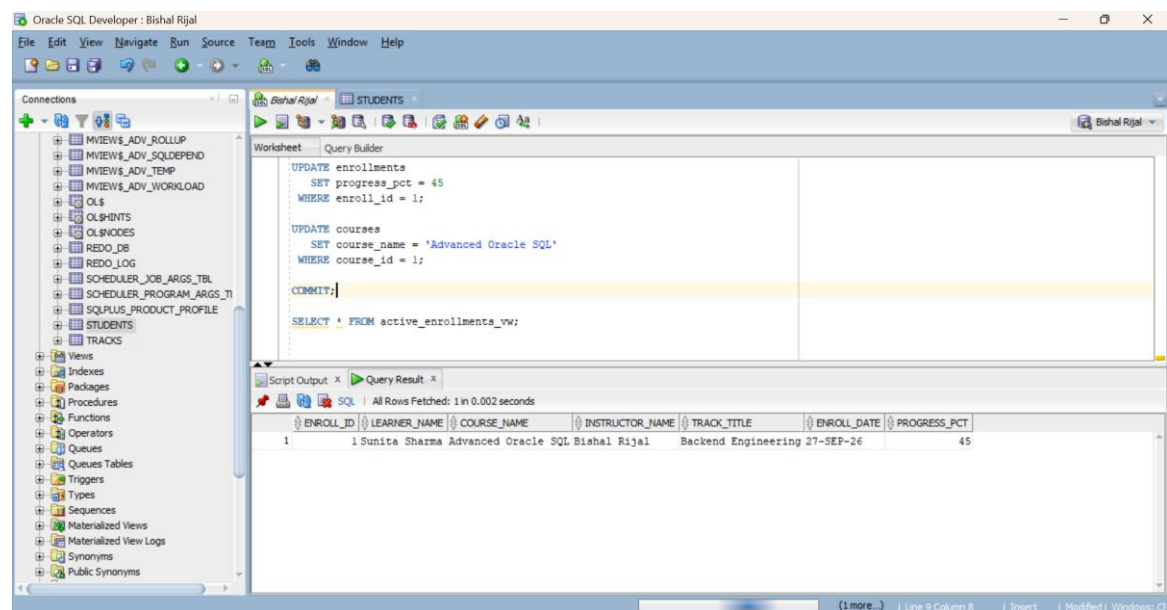


Fig: Update of COURSE_NAME and PROGRESS_PCT in ENROLLMENTS in COURSES, verified via the view.

Step 9: Call the Procedure

```

BEGIN

  drop_course(1);

END;

```

Step 10 : Verify the Procedure + Function Worked

```
SELECT * FROM audit_log;  
SELECT * FROM active_enrollments_vw;  
SELECT enroll_id, state FROM enrollments WHERE enroll_id = 1;
```

Output:

Query 1 — audit_log Output

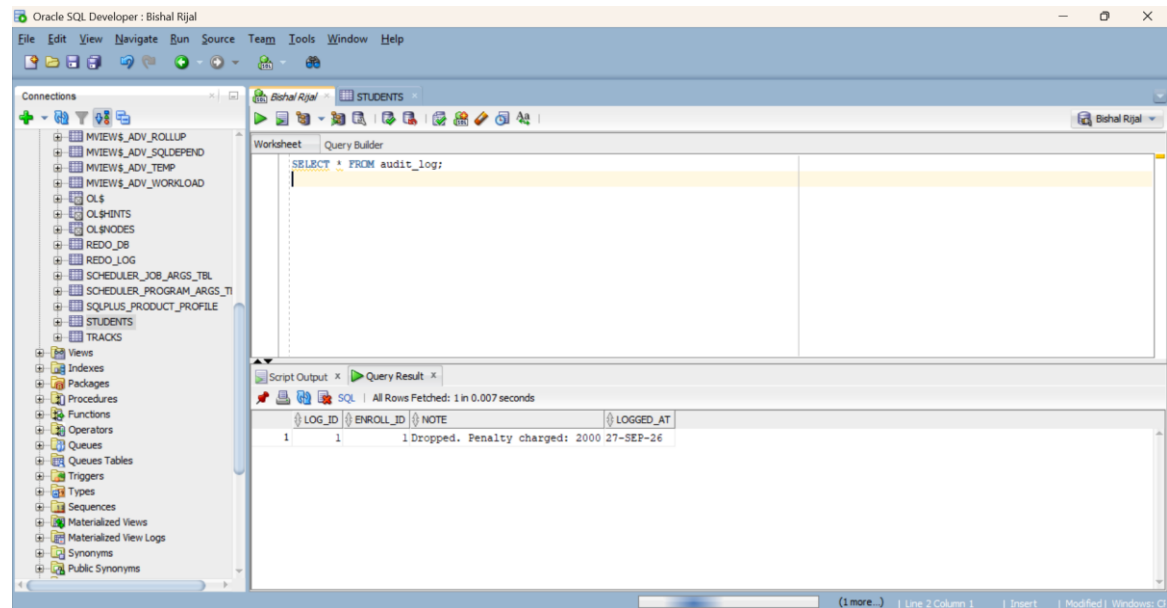


Fig: AUDIT_LOG output after executing DROP_COURSE(1).

Query 2 — active_enrollments_vw Output

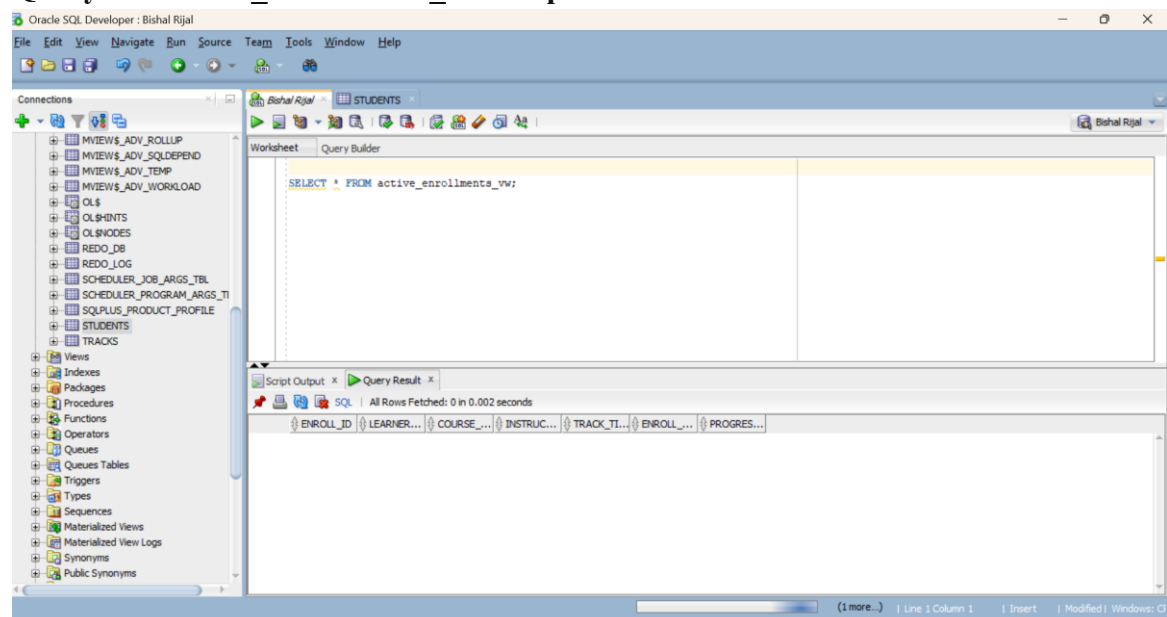


Fig: Output of ACTIVE_ENROLLMENTS_VW after withdrawal, showing zero ongoing enrollments.

Query 3 — enrollments row output

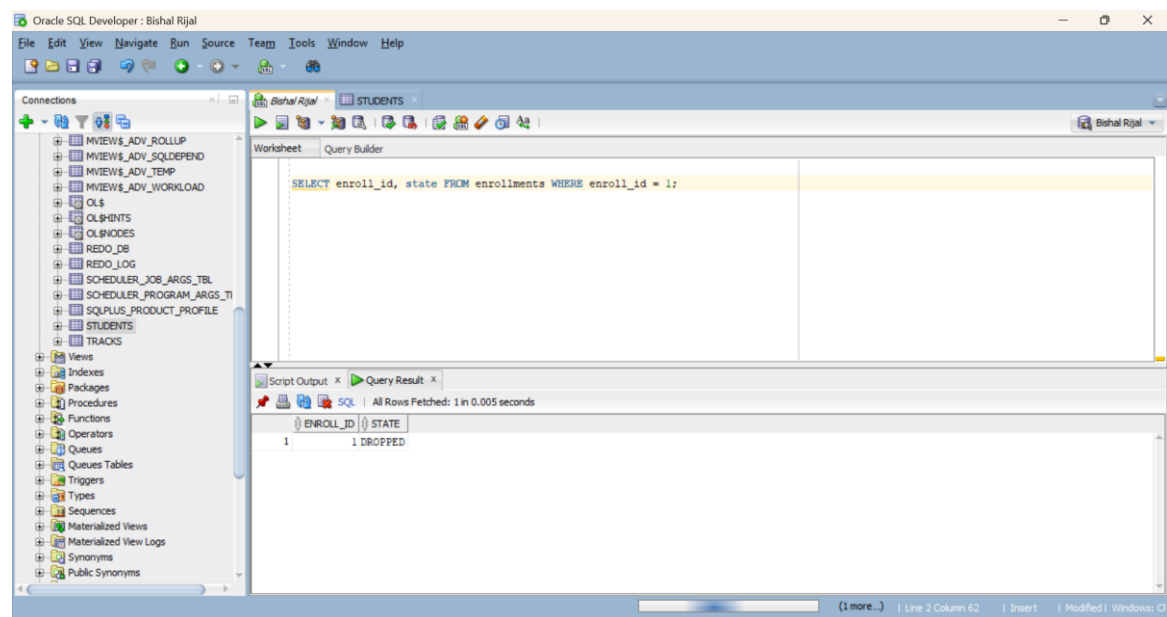


Fig: ENROLLMENTS record showing STATE = 'DROPPED'.

Step 11: Delete

```
DELETE FROM learners WHERE learner_id = 2;  
  
COMMIT;  
  
SELECT * FROM learners;
```

Output:

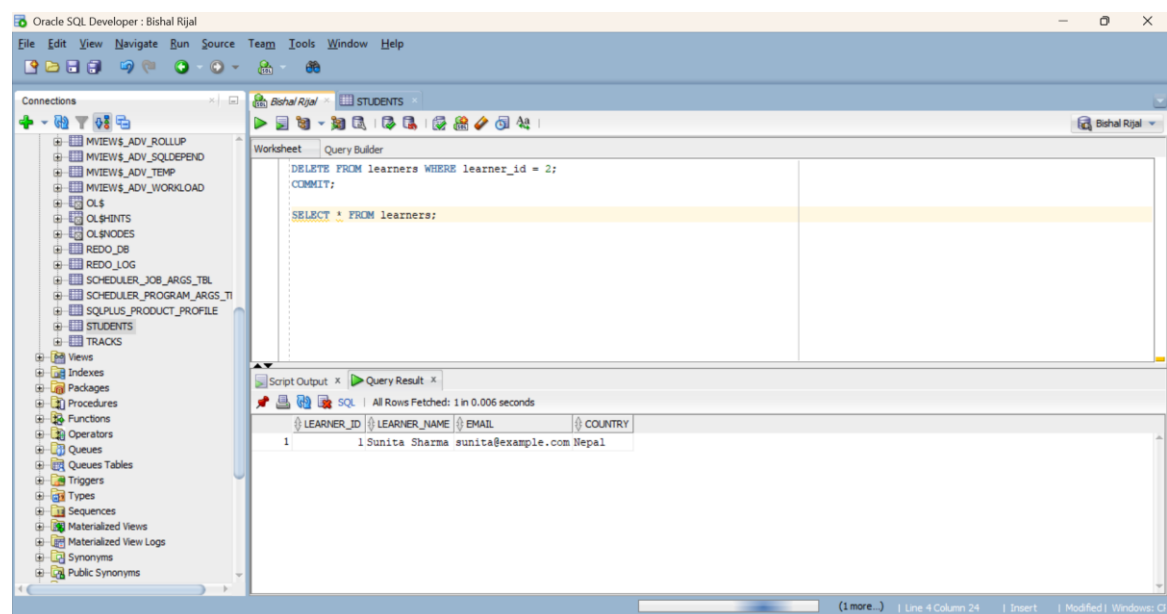


Fig: DELETE operation on LEARNERS and confirmation of remaining rows

Discussion:

The Online Course Enrollment System successfully combines the database concepts. The use of primary and foreign keys establishes relationships between learners, courses, instructors, and tracks, while constraints help maintain valid and consistent data.

The ACTIVE_ENROLLMENTS_VW provides a convenient way to retrieve information about ongoing enrollments without directly joining multiple tables in every query. The DROP_PENALTY() function calculates the withdrawal penalty based on the enrollment duration, while the DROP_COURSE() procedure performs the complete withdrawal operation and stores the activity in the audit log.

During development, database errors such as missing supporting objects or incorrect object dependencies may occur when creating procedures and functions. Creating the required tables and sequences before compiling dependent PL/SQL objects helps avoid such issues. Client-side validation can also be used together with database constraints to prevent invalid learner or enrollment information from being submitted.

Conclusion:

The Online Course Enrollment System successfully demonstrates the integration of relational tables, constraints, views, sequences, user-defined functions, stored procedures, and CRUD operations within a single database application. The project provides practical experience in designing a relational database and connecting its database objects with an application layer to manage online course enrollment activities efficiently.